A Project Report on

CAREERLINK
An AI-Integrated Campus Placement Portal

Submitted in partial fulfillment for award of

Bachelor of Technology

Degree

in

Computer Science and Engineering

By

Sk. Mohammad Ali (Y22ACS558) **M. Hema (Y22ACS501)**

V. Sravani (Y22ACS587) **S. Naga Goutham (Y22ACS564)**

Under the guidance of

Mr. K. Arun Babu, M.Tech

Assistant Professor

Department of Computer Science and Engineering

Bapatla Engineering College

(Autonomous)

(Affiliated to Acharya Nagarjuna University)

BAPATLA – 522 102, Andhra Pradesh, INDIA

2025-2026

Department of
Computer Science and Engineering

CERTIFICATE

This is to certify that the project report entitled **CareerLink: An AI-Integrated Campus Placement Portal** that is being submitted by Sk. Mohammad Ali (Y22ACS558), M. Hema Bhuvaneswari (Y22ACS501), V. Sravani (Y22ACS587), S. Naga Goutham (Y22ACS564) in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science & Engineering to the Acharya Nagarjuna University is a record of bonafide work carried out by them under our guidance and supervision.

Date:

Signature of the Guide	**Signature of the HOD**
Mr. K. Arun Babu	**Dr. M. Rajesh Babu**
Assistant Professor	**Assoc. Prof. & Head**

DECLARATION

We declare that this project work is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

Sk. Mohammad Ali (Y22ACS558)

M. Hema Bhuvaneswari (Y22ACS501)

V. Sravani (Y22ACS587)

S. Naga Goutham (Y22ACS564)

Acknowledgement

We sincerely thank the following distinguished personalities who have offered their valuable advice and support for the successful completion of this work.

We are deeply indebted to our most respected guide, **Mr. K. Arun Babu, Assistant Professor**, Department of Computer Science and Engineering, for his valuable guidance, constant encouragement, constructive suggestions, and inspiring support throughout the course of this project.

We extend our sincere thanks to **Dr. M. Rajesh Babu, Associate Professor & Head of the Department**, Department of Computer Science and Engineering, for his cooperation and for providing the necessary facilities and resources to carry out this work successfully.

We would also like to express our heartfelt gratitude to our beloved **Principal, Dr. B. Srinivasarao**, for providing the required infrastructure, online resources, and institutional support to complete this project.

We express our sincere thanks to our **Project Coordinators, Dr. C. Prakasa Rao, Professor, Department of CSE, and Dr. P. Venkata Lakshmi, Associate Professor, Department of CSE**, for their valuable suggestions and guidance in preparing and presenting this document.

We are always thankful to our **parents and family members** for their continuous support, encouragement, and motivation. Without their support, we would not have been able to complete this project work successfully.

Finally, we extend our sincere gratitude to all the teaching faculty and non-teaching staff of the Department of Computer Science and Engineering who helped us directly or indirectly with their cooperation and encouragement.

Sk. Mohammad Ali (Y22ACS558)

M. Hema Bhuvaneswari (Y22ACS501)

V. Sravani (Y22ACS587)

S. Naga Goutham (Y22ACS564)

ABSTRACT

Managing campus placements is a crucial activity for the placement cell of any engineering institution. The traditional manual approach, which relies on WhatsApp groups for communication and Google Forms for data collection, introduces significant inefficiencies: students frequently miss important job notifications, placement officers spend excessive time on administrative tasks, and there is no reliable mechanism for application tracking, data accuracy, or comprehensive placement reporting.

This project report presents CareerLink, a comprehensive AI-Integrated campus placement portal developed using the MERN technology stack (MongoDB, Express.js, React.js, Node.js). The system provides a unified platform for four stakeholder roles – students, recruiters, placement administrators, and alumni – each served through tailored role-based dashboards. Key innovations include an AI-powered Applicant Tracking System (ATS) using Google Gemini Flash 2.5 for semantic resume analysis and compatibility scoring, real-time WebSocket notifications via Socket.IO, automated account lifecycle management, an auto-shortlisting algorithm, bulk application processing, and an interactive alumni referral and career guidance module.

The platform was evaluated with 301 registered students across five engineering branches. The ATS mechanism achieves approximately 85% accuracy in matching candidate profiles with job requirements. Automation of repetitive administrative tasks results in a nearly 60% reduction in manual workload for placement staff. Real-time notifications are delivered with sub-500 ms latency. The system is deployed live: the frontend on Vercel and the backend on Render.

Keywords: *Campus Placement, Applicant Tracking System (ATS), Real-Time Notifications, WebSocket, Role-Based Dashboard, Google Gemini, MERN Stack, MongoDB, React.js, Node.js, Socket.IO, Cloudinary*

TABLE OF CONTENTS

Abstract v

Table of Contents vi

List of Tables ix

List of Figures x

List of Abbreviations xi

1 Introduction 1

1.1 Motivation 1

1.2 Problem Definition 1

1.3 Objectives of the Project 2

2 Literature Survey 4

2.1 Existing System 4

2.1.1 User Requirements 4

2.1.2 Software Requirements 4

2.1.3 Hardware Requirements 5

2.2 Disadvantages of Existing System 5

2.3 Proposed System 6

2.3.1 Student Module 6

2.3.2 Recruiter Module 7

2.3.3 Admin Module 8

2.3.4 Alumni Module 9

2.4 Advantages of the Proposed System 10

3 Analysis 11

3.1 Software Requirement Specification (SRS) 11

3.1.1 User Requirements 11

3.1.2 Software Requirements 12

3.1.3 Hardware Requirements 13

3.2 Context Diagram of the Project 14

4 Design 16

4.1 Introduction 16

4.2 System Design Diagrams 16

4.2.1 Three-Tier System Architecture 16

4.2.2 Use Case Diagram 17

4.2.3 Level-0 DFD (System Context Diagram) 18

4.2.4 Level-1 DFD – Job Application and ATS Scoring 19

4.2.5 Level-1 DFD – Notification Dispatch 20

4.2.6 Entity-Relationship (ER) Diagram 21

4.3 Data Dictionary 22

4.3.1 Collection Overview 22

4.3.2 Authentication Collection 23

4.3.3 Profile Collections 24

4.3.4 Placement Workflow Collections 30

4.3.5 Communication Collections 32

5 Implementation & Results 37

5.1 Method of Implementation 37

5.1.1 Frontend Implementation 37

5.1.2 Backend Implementation 38

5.1.3 AI-Powered ATS Implementation 39

5.1.4 Real-Time Notification Implementation 40

5.1.5 File Storage and Security 40

5.1.6 Deployment Architecture 41

5.2 Implementation of Algorithms 44

5.2.1 User Authentication with JWT 44

5.2.2 ATS Resume Scoring with Google Gemini AI 45

5.2.3 Auto-Shortlist Algorithm 47

5.2.4 Real-Time Notification System with Socket.IO 49

5.2.5 Bulk Application Status Update 51

5.3 Output Screens …… 53
5.3.1 Authentication Screens ……... 53
5.3.2 Student Module Screens ……. 54
5.3.3 Recruiter Module Screens …….. 56
5.3.4 Admin Module Screens …….. 58
5.3.5 Alumni Module Screens ……. 60
5.4 Result Analysis …… 60
5.4.1 Performance Metrics …….. 60
5.4.2 ATS Accuracy Validation …….. 61
5.4.3 Workload Reduction Estimation …… 62
6 Testing & Validation …….. 63
6.1 Features to be Tested ……... 63
6.2 Features Not to be Tested …… 63
6.3 Testing Tools and Environment …….. 64
6.4 Design of Test Cases and Scenarios …… 65
7 Conclusion & Future Enhancement ……. 68
7.1 Conclusion ……... 68
7.2 Future Enhancements …….. 68
7.2.1 Native Mobile Application ……. 69
7.2.2 Integrated Assessment Module …….. 69
7.2.3 External Platform Integration ……..... 69
7.2.4 Advanced AI Personalization ……. 69
8 References …….. 70

LIST OF TABLES

Table 3.1 Software Requirements Specification ……... 12

Table 3.2 Hardware Requirements …… 13

Table 4.1 CareerLink Database — Collection Overview …….. 22

Table 4.2 Data Dictionary — User Collection …….. 23

Table 4.3 Data Dictionary — Student Collection ……. 24

Table 4.4 Data Dictionary — Recruiter Collection ……... 27

Table 4.5 Data Dictionary — Admin Collection ……... 28

Table 4.6 Data Dictionary — Alumni Collection ….…… 29

Table 4.7 Data Dictionary — Job Collection …… 30

Table 4.8 Data Dictionary — Application Collection ……... 31

Table 4.9 Data Dictionary — Notification Collection …….. 32

Table 4.10 Data Dictionary — Post Collection ……... 33

Table 4.11 Data Dictionary — PostView Collection …….. 34

Table 4.12 Data Dictionary — Referral Collection ……. 35

Table 5.1 Full Technology Stack …….. 42

Table 5.2 Key REST API Endpoints ……. 43

Table 5.3 System Performance Metrics ……. 61

Table 6.1 Test Case Design and Results ……... 65

LIST OF FIGURES

Fig. 3.1	Context Diagram of CareerLink ……	14
Fig. 4.1	Three-Tier System Architecture Diagram …….	17
Fig. 4.2	Use Case Diagram — All Stakeholder Roles ……	18
Fig. 4.3	Level-0 DFD — CareerLink System Context ……...	19
Fig. 4.4	Level-1 DFD — Job Application and ATS Scoring Process ……	20
Fig. 4.5	Level-1 DFD — Notification Dispatch Process ……	21
Fig. 4.6	Entity-Relationship (ER) Diagram ……	22
Fig. 5.1	CareerLink Landing Page …….	53
Fig. 5.2	User Login Interface …….	53
Fig. 5.3	Student Dashboard ……	54
Fig. 5.4	Student Job Browsing Interface ……	54
Fig. 5.5	Real-Time Notification Delivery to Student …….	55
Fig. 5.6	ATS Match Score and Skill Gap Analysis Screen ……	55
Fig. 5.7	Recruiter Dashboard ……..	56
Fig. 5.8	Received Applications with ATS Scores ……..	56
Fig. 5.9	Auto-Shortlist Feature ……...	57
Fig. 5.10	Bulk Status Update ……	57
Fig. 5.11	Admin Analytics Dashboard …….	58
Fig. 5.12	Manage Students Interface — 301 Registered Students ……...	58
Fig. 5.13	Recruiter Approval ……	59
Fig. 5.14	Job Posting Moderation …….	59
Fig. 5.15	Alumni Dashboard ……	60
Fig. 5.16	Alumni Referral Posting Interface ……	60

LIST OF ABBREVIATIONS

AI	—	Artificial Intelligence
API	—	Application Programming Interface
ATS	—	Applicant Tracking System
bcrypt	—	Blowfish Crypt (password-hashing algorithm)
CDN	—	Content Delivery Network
CGPA	—	Cumulative Grade Point Average
CORS	—	Cross-Origin Resource Sharing
CSE	—	Computer Science and Engineering
CTC	—	Cost to Company
DFD	—	Data Flow Diagram
ECE	—	Electronics and Communication Engineering
EEE	—	Electrical and Electronics Engineering
ER	—	Entity-Relationship
ExcelJS	—	Excel JavaScript Library
FCM	—	Firebase Cloud Messaging
FR	—	Functional Requirement
HMR	—	Hot Module Replacement
HOD	—	Head of Department
HR	—	Human Resources
HTTP	—	Hypertext Transfer Protocol
JSON	—	JavaScript Object Notation
JWT	—	JSON Web Token
LLM	—	Large Language Model
LPA	—	Lakh Per Annum

MERN	—	MongoDB, Express.js, React.js, Node.js
NFR	—	Non-Functional Requirement
NoSQL	—	Not Only Structured Query Language
ODM	—	Object Document Mapper
PDF	—	Portable Document Format
REST	—	Representational State Transfer
SPA	—	Single Page Application
SRS	—	Software Requirement Specification
URL	—	Uniform Resource Locator
WebSocket	—	Web Socket Protocol
XLSX	—	Excel Spreadsheet (Office Open XML format)

1 INTRODUCTION

1.1 MOTIVATION

Campus placement is the primary gateway through which engineering graduates transition into professional careers. The placement cell of an institution plays a pivotal role in coordinating recruitment drives, managing student eligibility records, liaising with visiting companies, and ensuring equitable employment opportunities for all graduating students. With increasing student intake and a growing number of companies visiting campuses each academic year, the volume and complexity of placement-related activities have increased substantially.

Despite this growth, most engineering colleges continue to rely on informal, disconnected tools – WhatsApp groups, Google Forms, and Microsoft Excel spreadsheets – to manage the entire placement cycle. The result is a system that is slow, error-prone, and frustrating for all stakeholders. Students miss important deadlines buried in unrelated conversations, placement officers spend excessive time on administrative paperwork, and recruiters have no efficient mechanism to evaluate hundreds of applicants. This visible gap between the scale of modern campus placement activity and the inadequacy of the tools available motivated the development of CareerLink.

Additionally, the rapid advancement of AI-based language models, cloud infrastructure, and real-time communication technologies now makes it practical to build intelligent, scalable placement management systems that were previously out of reach for academic institutions. CareerLink leverages these technologies – specifically Google Gemini Flash 2.5 for AI-powered resume analysis, Socket.IO for real-time communication, and the MERN stack for full-stack development – to deliver a platform that genuinely transforms the placement experience.

1.2 PROBLEM DEFINITION

The existing manual campus placement management approach at Bapatla Engineering College, and at most engineering institutions, relies on WhatsApp groups, Google Forms, and Microsoft Excel spreadsheets. This approach introduces the following specific, well-documented problems:

- Notification gaps: Students must continuously monitor WhatsApp placement groups, where critical job notifications are frequently buried beneath unrelated messages. Deadlines pass before many students see the posting.
- No centralized job portal: There is no single platform where students can view all available job postings, filter by eligibility criteria, or track their application history across companies.
- Manual and error-prone data collection: Placement officers create a new Google Form for every company visit. Manual data entry does not guarantee accuracy, and duplicate or inconsistent submissions are common.
- No automated candidate screening: Recruiters must manually read through every form submission to identify eligible candidates, with no ranking or scoring mechanism to assist the process.
- Excel-based record management: Student records maintained across multiple spreadsheets are difficult to synchronize, prone to corruption, and impossible to query in real time during peak placement season.
- Laborious report generation: End-of-year placement statistics – placement percentage, average CTC, branch-wise and company-wise performance – must be compiled manually from dozens of spreadsheets, requiring days of effort with high risk of error.
- No alumni integration: There is no platform mechanism to leverage alumni networks for referral-based job opportunities or career guidance sharing with current students.
- Lack of transparency: Students have no visibility into the status of their applications after submission, and placement officers lack a real-time dashboard to monitor the placement pipeline.

1.3 OBJECTIVES OF THE PROJECT

The primary objective of this project is to design and develop CareerLink, a centralized, AI-powered campus placement portal that eliminates the inefficiencies of the existing manual system. The specific objectives are:

1. To build a secure, full-stack web application using the MERN stack serving four distinct user roles – students, recruiters, placement administrators, and alumni – through a unified platform with role-based access control.

2. To implement an AI-powered Applicant Tracking System (ATS) using Google Gemini Flash 2.5 for automated semantic resume analysis, ATS compatibility scoring (0–100), strength identification, and skill gap analysis.
3. To deliver real-time, targeted notifications to all stakeholders using Socket.IO WebSocket communication, ensuring sub-500 ms delivery for events such as job approvals, application status changes, and referral postings.
4. To automate critical administrative workflows including student account lifecycle management (year-end expiry and extension), job approval pipelines, bulk application status updates for 200+ applications simultaneously, and filtered Excel report generation.
5. To provide comprehensive placement analytics dashboards with branch-wise and company-wise statistics, enabling data-driven decision-making by placement administrators.
6. To include an alumni referral and feed module that allows verified alumni to post job referral opportunities and career guidance, strengthening the connection between alumni and current students.
7. To deploy the application as a production-ready system, with the frontend hosted on Vercel and the backend on Render, accessible to real users at Bapatla Engineering College.

2 LITERATURE SURVEY

2.1 EXISTING SYSTEM

Several research groups have proposed web-based solutions to address the challenges of campus placement management. Three representative works from 2022 and 2023 are reviewed below. Each system addresses certain aspects of the problem but leaves critical gaps that CareerLink is specifically designed to resolve.

2.1.1 Institutional Recruitment Tool – Raj et al. (2022)

Raj et al. [1] developed an institutional recruitment tool for students and placement cells that automates student filtering based on eligibility criteria and generates placement reports. The system introduces a structured approach to managing recruitment within academic institutions, replacing manual filtering with automated eligibility checks based on predefined criteria such as CGPA thresholds and branch requirements. The platform also generates basic placement reports at the end of a recruitment cycle, reducing some of the manual effort involved in data compilation.

However, the system relies entirely on email-based notifications for communication, which introduces delays and depends on students actively monitoring their inboxes. There is no provision for real-time communication, meaning that time-sensitive updates during active placement drives may not reach students promptly. The system also lacks recruiter-side dashboards, ATS-based resume scoring, alumni integration, and centralized application tracking.

2.1.2 Training and Placement Portal – D’Monte et al. (2023)

D'Monte et al. [2] proposed a training and placement portal that digitizes the recruitment process for college students. The system supports student profile management, resume building within the platform, and aptitude test preparation modules. It takes a student-centric approach to placement readiness, going beyond recruitment coordination to include preparation resources.

While this system represents a meaningful improvement over manual methods, it does not incorporate automated resume screening or ATS-based candidate evaluation. Recruiters using this platform must still manually review all resumes to identify the

best-fit candidates – the most time-consuming part of the recruitment process. The system also lacks real-time notification delivery, a recruiter-facing dashboard, alumni referral management, and automated administrative workflows.

2.1.3 MERN Stack Job Portal – Srivastava et al. (2023)

Srivastava et al. [3] developed a MERN stack-based job portal for in-campus placements that allows students to browse and apply for job opportunities posted by the placement department. The system is built on modern web technologies – MongoDB, Express.js, React, and Node.js – providing a technically sound foundation. It offers a centralized interface for job management and student-facing job browsing.

Despite its technical foundation, the system is primarily a one-way information platform. It lacks recruiter-side functionality, meaning companies cannot independently post jobs or manage their application pipelines. There is no alumni integration, no real-time WebSocket notification delivery, no ATS-based resume scoring, and no automated account lifecycle management.

2.2 DISADVANTAGES OF EXISTING SYSTEM

The traditional manual approach – using WhatsApp, Google Forms, and Excel – and the reviewed digital systems share a common set of critical limitations that CareerLink is designed to overcome:

- Absence of real-time communication: All reviewed systems use either manual broadcasting or email-based notifications, both of which are slow and unreliable during active placement seasons.
- No AI-powered candidate screening: None of the reviewed systems incorporate automated resume analysis, ATS compatibility scoring, or skill gap identification.
- Incomplete stakeholder coverage: No reviewed system provides dedicated dashboards and workflows for all four stakeholder types – students, recruiters, administrators, and alumni – within a single integrated platform.
- No alumni ecosystem: Alumni referral management and structured knowledge sharing between alumni and current students are entirely absent in all reviewed systems.

- Persistent manual administrative burden: Account management, job approval workflows, bulk application processing, and report generation remain manual processes in all existing approaches.
- No career guidance for unplaced students: None of the existing systems provide any form of personalized support or guidance for students who do not secure placement.
- No auto-shortlisting: The ability to automatically select top candidates based on an ATS score threshold is absent in all reviewed systems.
- No production deployment: The reviewed systems were built as research prototypes without live deployment, real user testing, or institutional adoption.

2.3 PROPOSED SYSTEM

CareerLink is proposed as a comprehensive, production-deployed solution to the limitations identified in Sections 2.1 and 2.2. It is a full-stack MERN web application augmented with AI capabilities, real-time communication, and cloud infrastructure. The system is designed around four primary modules, each serving a distinct stakeholder group within the campus placement ecosystem.

2.3.1 Student Module

In this module, students initially register through a unique registration number the college provides and log in to the platform using registration number or email along with a password. Students must complete their profile that includes academic details, resume, projects, internships, certifications, and external links when they first log in. Students can update their details, and the changes are reflected in their profiles. Whenever a student uploads a resume, the system parses the resume and uses the extracted information for Applicant Tracking System (ATS) processing. The system stores all this information in the database.

Whenever a company posts a job or internship, the notification system sends instant alerts to eligible students, allowing them to browse and apply with a single click. During application submission, the system calculates an ATS score by comparing the resume text with the job description. This results in displaying an overview of the candidate's strengths and weaknesses so that they can address the identified skill gaps.

Whenever a placement officer or recruiter updates the status of an application, students receive instant notifications regarding the application status. This reduces the time required in manual methods, where recruiters used to send the data to the placement officer, who would then post it in placement groups, and students had to review the data to know their status. Alumni can also post job referrals on the platform. The system notifies students about new referrals, and they can apply through the provided link.

Furthermore, the system introduces an automated account lifecycle. At the end of the academic year, the system expires and deletes student accounts from the database to reduce the load on the system database. Students placed in companies have dashboards that reflect their respective companies and provide relevant guidance and recommendations. For students who remain unplaced, the dashboard offers career guidance and an option to extend their accounts.

2.3.2 Recruiter Module

Recruiters can register on the platform and must await approval from the administrator before gaining full access. After successful approval, recruiters can log in and complete their profile by providing essential company information. When a recruiter posts a job opportunity, the system notifies the administrator for job verification and approval. Once approved, the job posting becomes visible to students, allowing them to apply through the platform. Both the recruiter and eligible students receive notifications regarding the job posting.

Recruiters can review the received applications through the system. Applications are displayed along with the calculated Applicant Tracking System (ATS) score, which helps recruiters evaluate candidates efficiently. In addition to ATS-based evaluation, recruiters can manually analyse student resumes. The system also provides an auto-shortlisting feature, where recruiters can specify a minimum ATS score to automatically shortlist eligible candidates.

The platform also allows recruiters to post updates related to the placement process. These updates are displayed in the student feed section, and students receive notifications about the new information.

Recruiters can update the status of applications during different stages of the recruitment process, such as shortlisting, interview rounds, and final selection. For bulk updates, recruiters can provide a list of student registration numbers to update multiple applications simultaneously. Whenever the application status is updated, students receive real-time notifications through the platform. At the end of the recruitment process, the system enables recruiters to export the list of selected candidates or all applicants for documentation and reporting purposes.

2.3.3 Admin Module

This module provides administrators with a centralized dashboard that enables them to monitor platform activities, manage users, and control recruitment workflows.

When a new recruiter or alumni registers on the platform, the administrator receives a notification. The administrator can review the registration details and either approve or reject the account. Administrators are also responsible for managing student accounts and can extend account validity when required.

When recruiters post new job opportunities or alumni submit referral opportunities, these entries are first reviewed by the administrator before becoming visible to students. This approval mechanism prevents the circulation of inaccurate or unauthorized information and ensures that only verified opportunities are made available to students.

The analytics dashboard provides administrators with a comprehensive overview of placement activities. It enables them to analyse placement statistics such as branch-wise and company-wise selections. The export feature allows administrators to download filtered data based on criteria such as branch, company, or date range, which assists in preparing placement reports for college management.

Administrators can also post announcements on the platform feed that are visible to all students. Students receive notifications for these announcements, which may include information about upcoming placement drives, important deadlines, or policy updates. Important posts can be pinned so that they remain visible at the top of the feed.

The administrator can also manage application workflows by reviewing student applications, updating application statuses, and exporting lists of selected candidates or all applicants on a company-wise basis. In addition to regular administrators, the system includes a Super Admin role with higher-level privileges. The Super Admin is responsible for managing administrative accounts and ensuring proper system governance. This role allows the creation, management, and removal of other admin accounts, enabling better control over administrative responsibilities within the platform.

Through these functionalities, the Admin Module ensures effective platform governance, enhances transparency, and supports efficient coordination between students, recruiters, alumni, and placement teams.

2.3.4 Alumni Module

The Alumni Module enables former students of the institution to contribute to the placement ecosystem by sharing industry opportunities and career guidance with current students. Alumni can register on the platform using their academic credentials, including registration number, branch, and graduation year. After registration, the account awaits verification by the administrator to ensure authenticity. Once approved, alumni can log in and complete their profile by providing details about their current company, designation, and professional experience.

A key feature of this module is the referral system, which allows alumni to post job referral opportunities available in their organizations. When an alumni member submits a referral opportunity, it is first reviewed by the administrator before being published on the platform. After approval, the referral becomes visible to students, and students receive notifications about the new opportunity along with the referral link provided by the alumni.

In addition to referrals, alumni can share career-related updates, insights, or guidance through the platform feed. These posts appear in the student feed section and help students stay informed about industry trends, hiring processes, and professional development opportunities.

Through these functionalities, the Alumni Module strengthens the connection between alumni and students, enabling knowledge sharing and increasing access to referral-based job opportunities, thereby supporting the overall placement process.

2.4 ADVANTAGES OF THE PROPOSED SYSTEM

CareerLink's proposed design offers the following key advantages over the existing manual approach and all reviewed existing systems:

- Real-time WebSocket notifications via Socket.IO eliminate information delays entirely, delivering critical updates to the right users within 500 ms.
- AI-powered ATS scoring using Gemini Flash 2.5 automates resume screening with semantic understanding, replacing hours of manual recruiter effort with consistent, objective compatibility scoring.
- Role-based dashboards for all four stakeholder groups provide each user type with precisely the tools they need within a single, integrated platform.
- Auto-shortlisting with configurable ATS threshold eliminates the most tedious part of the recruiter workflow and ensures that the most qualified candidates are prioritized consistently.
- Bulk application processing allows status updates for 200+ applications in under 2 seconds.
- Alumni referral integration expands student placement opportunities beyond direct company campus visits.
- Automated account lifecycle management ensures database hygiene across academic years without any manual intervention.
- Centralized analytics dashboards provide instant, accurate placement statistics, completely eliminating the need for days of manual report compilation.
- Career guidance for unplaced students ensures that every graduating student receives support and direction.
- Production deployment on Vercel and Render ensures real-world availability, scalability, and institutional adoption.

3 ANALYSIS

3.1 SOFTWARE REQUIREMENT SPECIFICATION (SRS)

3.1.1 User Requirements

The following user requirements were identified through structured stakeholder analysis involving placement officers, a sample of students, and a recruiter representative:

3.1.1.1 Student Requirements

- Register securely using a college-assigned registration number; log in using registration number or email.
- Build and maintain a comprehensive academic and professional profile including CGPA, branch, skills, projects, internships, certifications, and external links.
- Upload a resume in PDF format (maximum 5 MB) and receive automated ATS analysis and skill gap feedback upon each job application.
- Browse all available, approved job and internship postings automatically filtered to personal eligibility criteria.
- Apply for eligible postings with a single click and track application status updates in real time.
- Receive instant WebSocket notifications for new job postings, referrals, application status changes, and platform announcements.
- Access career guidance recommendations and, if unplaced, request an account extension at year-end.

3.1.1.2 Recruiter Requirements

- Register with company details and await administrator verification before accessing full platform features.
- Post job and internship opportunities with complete eligibility criteria including branch filter, CGPA cutoff, backlog policy, required skills, CTC, and application deadline.
- View all received applications ranked by ATS compatibility score; access individual student resumes via Cloudinary URLs.
- Set an ATS score threshold to auto-shortlist the top-performing candidates automatically.

- Update application statuses individually or in bulk via CSV upload; receive confirmation notifications.
- Post recruitment process updates to the student feed; export applicant lists as XLSX files.

3.1.1.3 Administrator Requirements

- Approve or reject recruiter and alumni registrations; manage student account extensions and year-end lifecycle expiry.
- Review and approve or reject job postings and alumni referrals before they are visible to students.
- Monitor platform activity through a real-time analytics dashboard with branch-wise and company-wise placement statistics.
- Generate and export filtered placement reports (by branch, company, or date range) as Excel files.
- Post and pin platform-wide announcements; manage feed content.
- Super Admin: Create, manage, and remove other administrator accounts.

3.1.1.4 Alumni Requirements

- Register using academic credentials (registration number, branch, graduation year) and await administrator verification.
- Complete a professional profile with current company, designation, and experience details.
- Submit job referral opportunities from current employer; track approval status and post publication confirmation.
- Share career guidance, interview tips, and industry insights through the platform feed.

3.1.2 Software Requirements

The following table details the complete software requirements for the CareerLink system:

Table 3.1: Software Requirements Specification

Component	Specification
Operating System	Windows 10/11, macOS 12+, Linux Ubuntu 20.04+ (server or client)
Client Browser	Google Chrome 110+, Mozilla Firefox 110+, Microsoft

	Edge 110+
Frontend Framework	React.js 18.2.0 (with Vite 5.0 build tool and HMR)
Styling	Tailwind CSS 3.3 with PostCSS
HTTP Client	Axios 1.6+ (with JWT request/response interceptors)
Real-Time Client	Socket.IO Client 4.5+
Charts	Recharts 2.8+ for analytics visualizations
Routing	React Router v6
Backend Runtime	Node.js 18.x LTS
Backend Framework	Express.js 4.18+
ODM	Mongoose 8.0+
Database	MongoDB 6.0+ (MongoDB Atlas cloud-hosted)
Authentication	JSON Web Token (jsonwebtoken) + bcrypt 5.1+
AI Service	Google Gemini Flash 2.5 API
File Storage	Cloudinary SDK 1.40+
Security	Helmet 7.1+, CORS 2.8+, express-rate-limit
Validation	express-validator 7.0+
Excel Export	ExcelJS 4.3+
Environment	dotenv 16.3+
Process Manager	PM2 (production server management)

3.1.3 Hardware Requirements

The following table details the hardware requirements for deploying and accessing CareerLink:

Table 3.2: Hardware Requirements

Component	Specification	Remarks
Processor	Intel Core i3 / AMD Ryzen 3 or above	For running dev server and MongoDB locally
RAM	Minimum 4 GB (8 GB recommended)	Node.js server + local MongoDB instance
Storage	Minimum 10 GB free	Project files, node_modules, local DB
Network	Broadband / Wi-Fi (min. 2 Mbps)	Gemini API calls, Cloudinary uploads

Client Device	Any PC, laptop, tablet, or smartphone	Browser-based access to the web app
MongoDB Hosting	MongoDB Atlas M0 Free Tier or above	Cloud database for production
Backend Hosting	Render Free/Starter Tier	Node.js server deployment
Frontend Hosting	Vercel Hobby / Pro Tier	React SPA with CDN

3.2 CONTEXT DIAGRAM OF THE PROJECT

The context diagram (Fig. 3.1) represents CareerLink as a single processing system node that interacts with four external entities. All information flows between external entities and the CareerLink system pass through well-defined interfaces, providing a high-level view of the system boundaries.

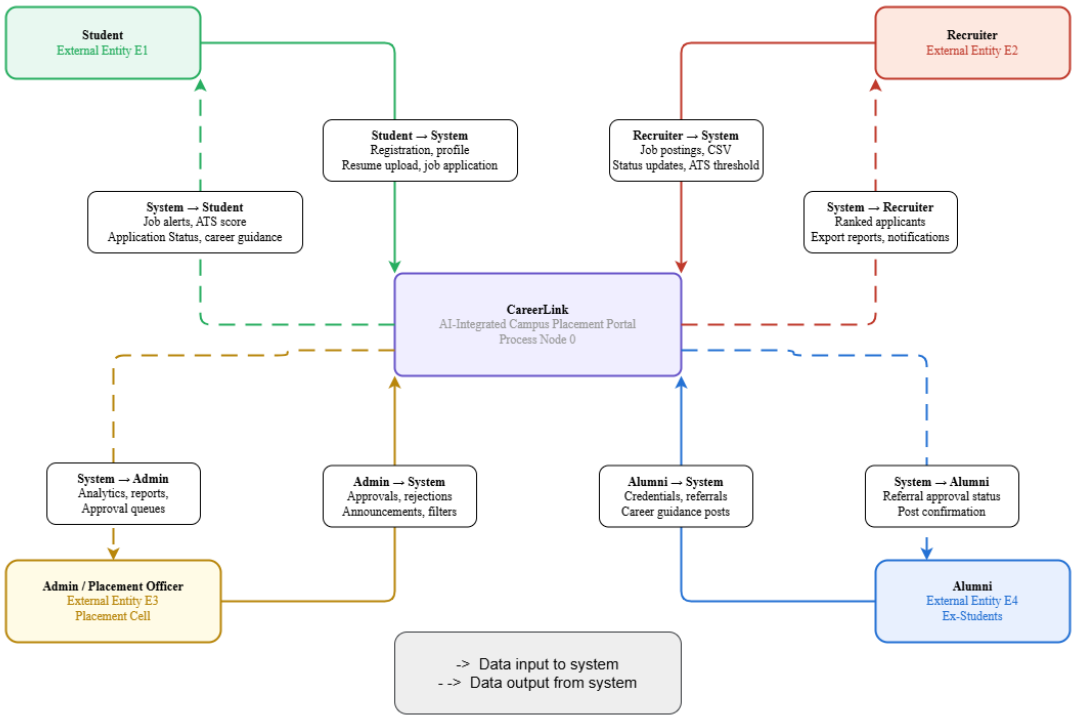


Fig. 3.1: Context Diagram of CareerLink

Student ↔ CareerLink: Inputs – registration data, profile updates, resume upload, job application request. Outputs – job notifications, ATS score and skill gap report, application status updates, career guidance, alumni referral alerts.

Recruiter ↔ CareerLink: Inputs – company registration, job posting data, application status updates, bulk CSV, ATS threshold. Outputs – approval notification, applicant list ranked by ATS, export files, student notifications.

Administrator ↔ CareerLink: Inputs – account approval decisions, job verification, announcement content, export filters. Outputs – platform analytics, placement reports, pending approval queues, system health status.

Alumni ↔ CareerLink: Inputs – academic credentials for registration, referral opportunity data, career guidance posts. Outputs – verification status, referral approval notification, post publication confirmation.

4 DESIGN

4.1 INTRODUCTION

The design phase translates the functional and non-functional requirements identified in Chapter 3 into a concrete architectural and structural blueprint for system implementation. CareerLink's design is guided by the principles of modularity, separation of concerns, scalability, and security. The design artefacts presented in this chapter include the three-tier system architecture, use case diagram, data flow diagrams at Level-0 and Level-1, entity-relationship (ER) diagram, and a comprehensive data dictionary for all primary database collections.

4.2 SYSTEM DESIGN DIAGRAMS

4.2.1 Three – Tier System Architecture

CareerLink implements a three-tier client-server architecture that cleanly separates the user interface, business logic, and data storage concerns:

- Tier 1 – Presentation Layer: The React.js 18 Single Page Application [4] served to the user's browser via Vercel's CDN. Built with Vite 5.0 for fast development builds with Hot Module Replacement. Tailwind CSS 3.3 [11] provides utility-first responsive styling. Role-specific dashboards are rendered conditionally based on the authenticated user's role stored in the AuthContext.

- Tier 2 – Application Layer: The Node.js 18 LTS server [5] running Express.js 4.18 [6] deployed on Render. Processes all business logic through a modular controller architecture. Exposes RESTful APIs for all CRUD operations. Socket.IO 4.5 server runs on the same HTTP instance for real-time bidirectional communication. JWT middleware protects all authenticated routes; roleAuth middleware enforces per-role access control.

- Tier 3 – Data Layer: MongoDB 6.0 hosted on MongoDB Atlas [7] stores all persistent data across 11 collections. Mongoose 8.0 provides schema validation and ODM capabilities. Cloudinary CDN stores all binary files (resumes and profile photos), with only secure URLs stored in MongoDB. Google Gemini Flash 2.5 API provides AI inference on demand for ATS scoring per application.

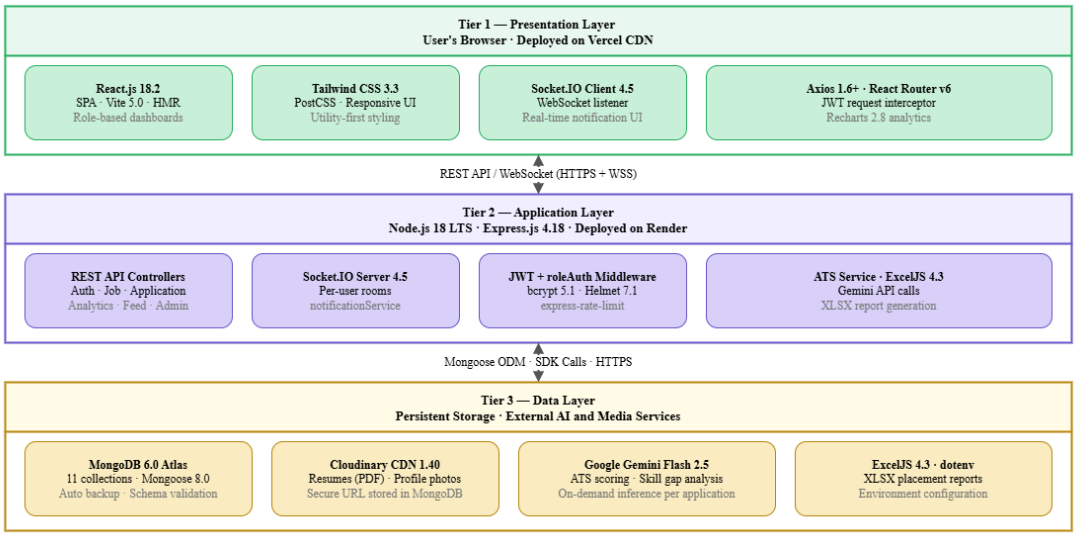


Fig. 4.1: Three-Tier System Architecture Diagram

4.2.2 Use Case Diagram

The use case diagram (Fig. 4.2) captures all interactions between the four actor types and the system's primary use cases. Student use cases include: Register, Login, Manage Profile, Upload Resume, Browse Jobs, Apply for Job (triggers ATS scoring), View ATS Score and Skill Gap Report, Track Application Status, Receive Notifications, View Alumni Feed, and Request Account Extension. Recruiter use cases include: Register, Login, Post Job (enters admin approval queue), View ATS-Ranked Applications, Auto-Shortlist Candidates, Bulk Update Application Status, Export Applicant List, and Post Feed Update. Admin use cases include: Approve/Reject Accounts (recruiters and alumni), Approve/Reject Job Postings, Approve/Reject Alumni Referrals, View Analytics Dashboard, Export Placement Reports, Post and Pin Announcements, Manage Student Accounts, and Manage Admin Accounts (Super Admin). Alumni use cases include: Register, Login, Post Referral Opportunity, and Post Career Guidance to Feed.

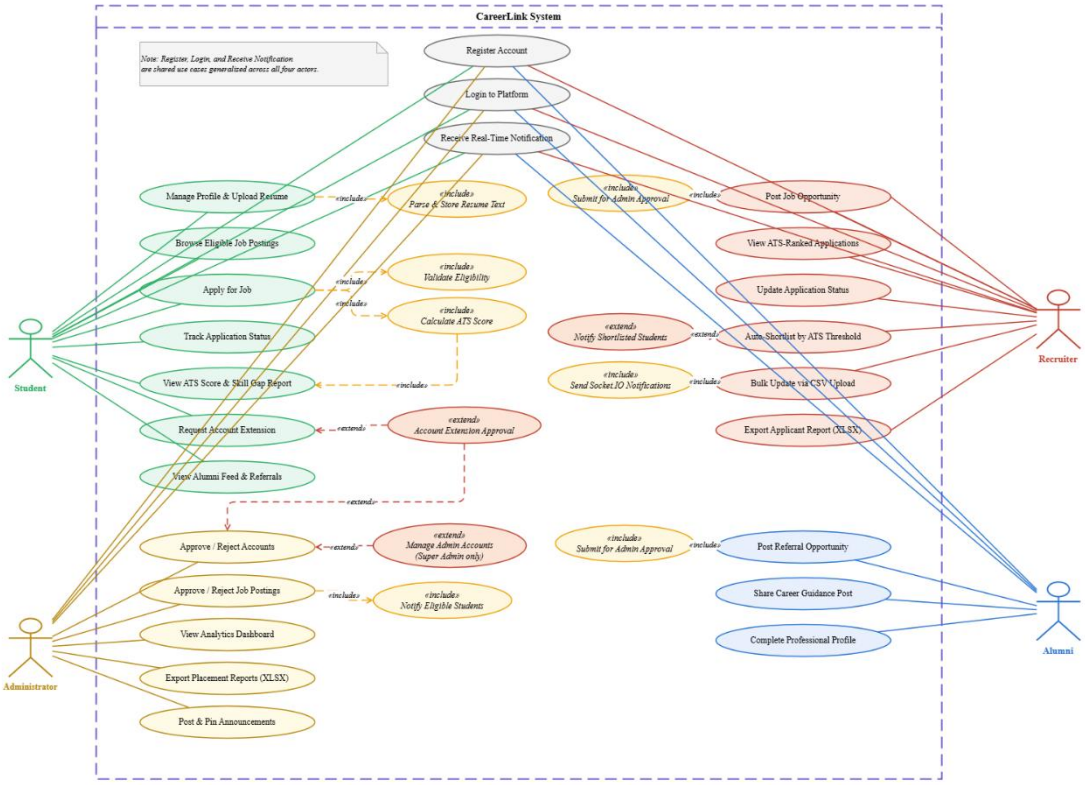


Fig. 4.2: Use Case Diagram – All Stakeholder Roles

4.2.3 Level – 0 DFD (System Context Diagram)

The Level-0 DFD (Fig. 4.3) represents the entire CareerLink system as a single process node. Four external entities interact with the system through defined data flows. Student data flows include: inputs (registration, profile, resume, application request) and outputs (notifications, ATS report, status updates, career guidance). Recruiter data flows include: inputs (registration, job postings, status updates, CSV) and outputs (approval, applicant list, export file). Admin data flows include: inputs (approval decisions, announcements, export queries) and outputs (analytics, reports, approval queues). Alumni data flows include: inputs (registration, referral data, career posts) and outputs (verification, publication confirmations).

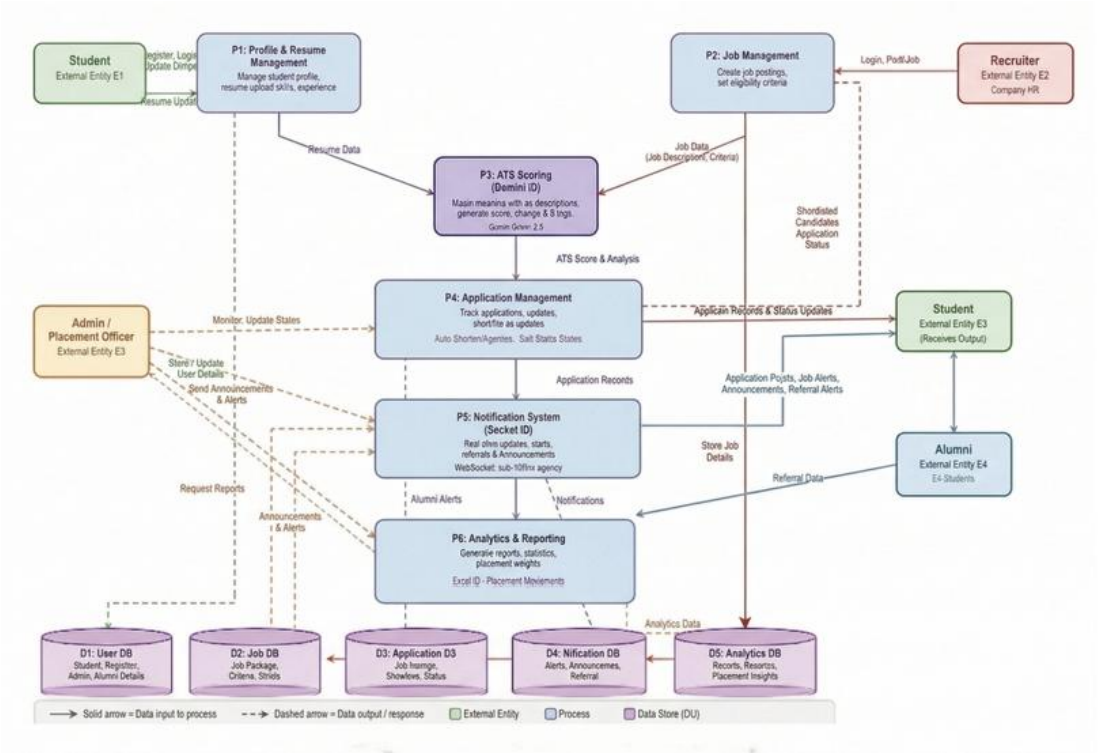
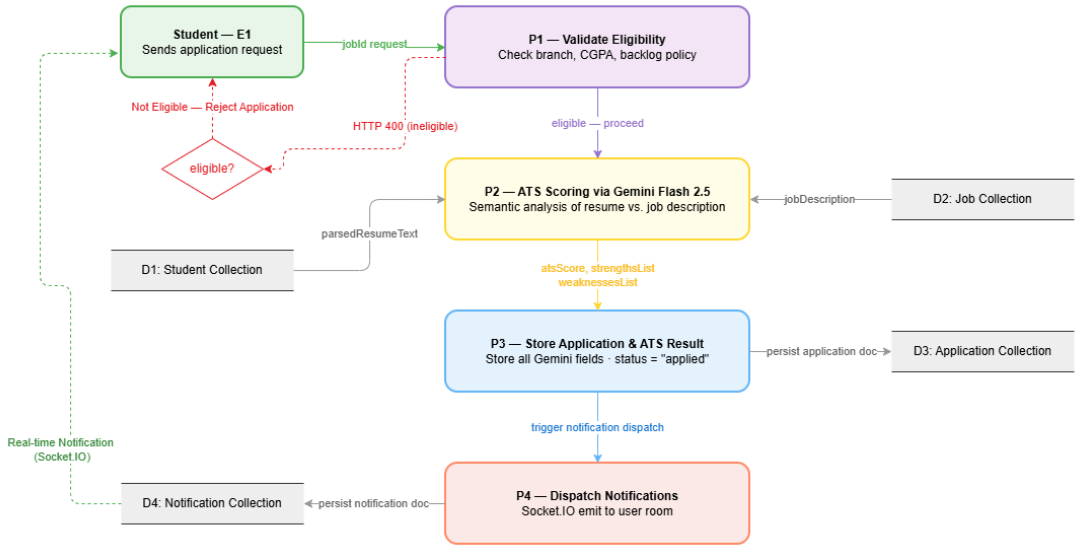


Fig. 4.3: Level-0 DFD – CareerLink System Context

4.2.4 Level – 1 DFD – Job Application and ATS Scoring

The Level-1 DFD (Fig. 4.4) decomposes the job application and ATS scoring process into sequential sub-processes: (1) Student submits application request to the Application Controller via REST API. (2) Application Controller queries the Job Collection to verify student eligibility against branch, CGPA, and backlog criteria. (3) If ineligible, an error response is returned to the student; process terminates. (4) If eligible, parsedResumeText is retrieved from the Student Collection and jobDescription from the Job Collection. (5) Both texts are transmitted to the Google Gemini Flash 2.5 API for semantic analysis. (6) Gemini returns atsScore, strengthsList, weaknessesList, and overallSummary in JSON format. (7) A new Application document is created with all Gemini output fields and initial status of 'applied'. (8) A notification is dispatched to the student and recruiter via Socket.IO confirming successful application submission.



Gemini API Response Fields		
Field	**Type**	**Description**
atsScore	Float (0–100)	Compatibility score between resume and job description
strengthsList	Array<String>	Matched skills between candidate and job requirements
weaknessesList	Array<String>	Skill gaps identified relative to job requirements

Fig. 4.4: Level-1 DFD – Job Application and ATS Scoring Process

4.2.5 Level – 1 DFD – Notification Dispatch

When any backend trigger event occurs (job approval, status change, referral publication, announcement), the Notification Controller: (1) creates a Notification document in MongoDB with the event type, recipient userId, message text, and read = false; (2) retrieves the active socket mapping for the target userId from the Socket.IO server's in-memory store; (3) emits a targeted Socket.IO event to the recipient's room (user:${userId}); (4) the client-side NotificationContext listener receives the event and updates the unread count badge and notification dropdown without any page refresh. If the user has granted browser notification permission, a native desktop push notification is also triggered via the browser Notification API.

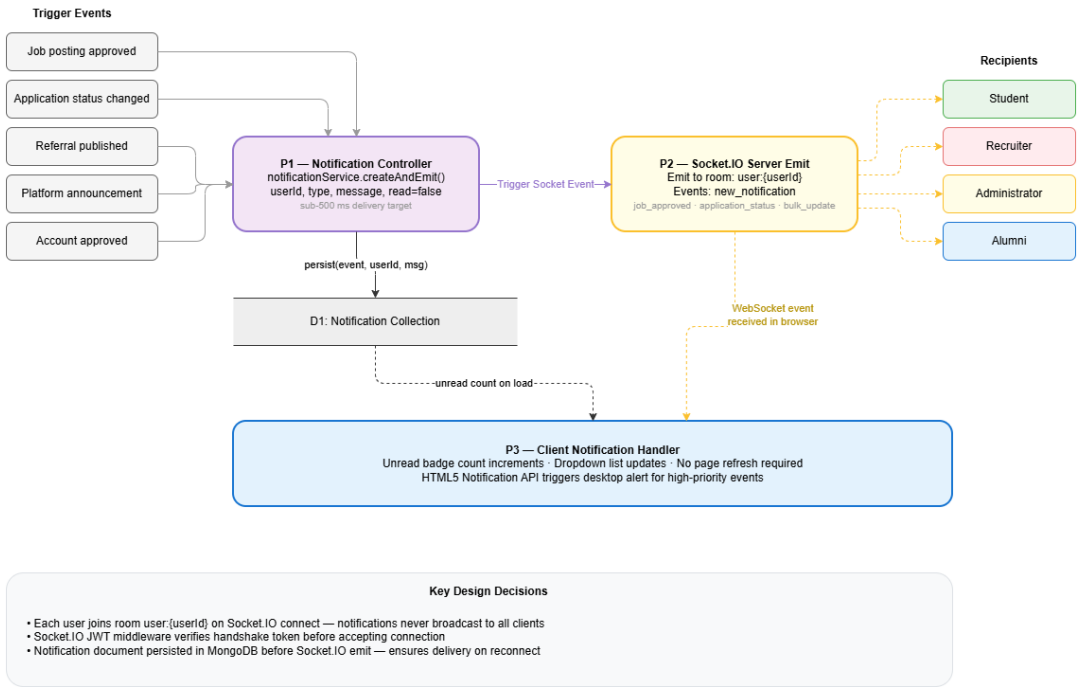


Fig. 4.5: Level-1 DFD – Notification Dispatch Process

4.2.6 Entity – Relationship (ER) Diagram

The ER diagram (Fig. 4.6) models the data entities and their relationships within the CareerLink MongoDB database. Primary entities and their cardinality relationships:

- User (1) ↔ (1) Student / Recruiter / Admin / Alumni: Each user account is linked to exactly one role-specific profile document.
- Student (1) ↔ (M) Application: A student may submit applications to multiple job postings.
- Job (1) ↔ (M) Application: A job posting may receive applications from multiple students.
- Recruiter (1) ↔ (M) Job: A recruiter account may post multiple job opportunities.
- Alumni (1) ↔ (M) Referral: An alumni account may post multiple referral opportunities.
- Admin (1) ↔ (M) Post: An administrator account may create multiple platform announcements.

- User (1) ↔ (M) Notification: A user may have multiple notification records in the Notification collection.

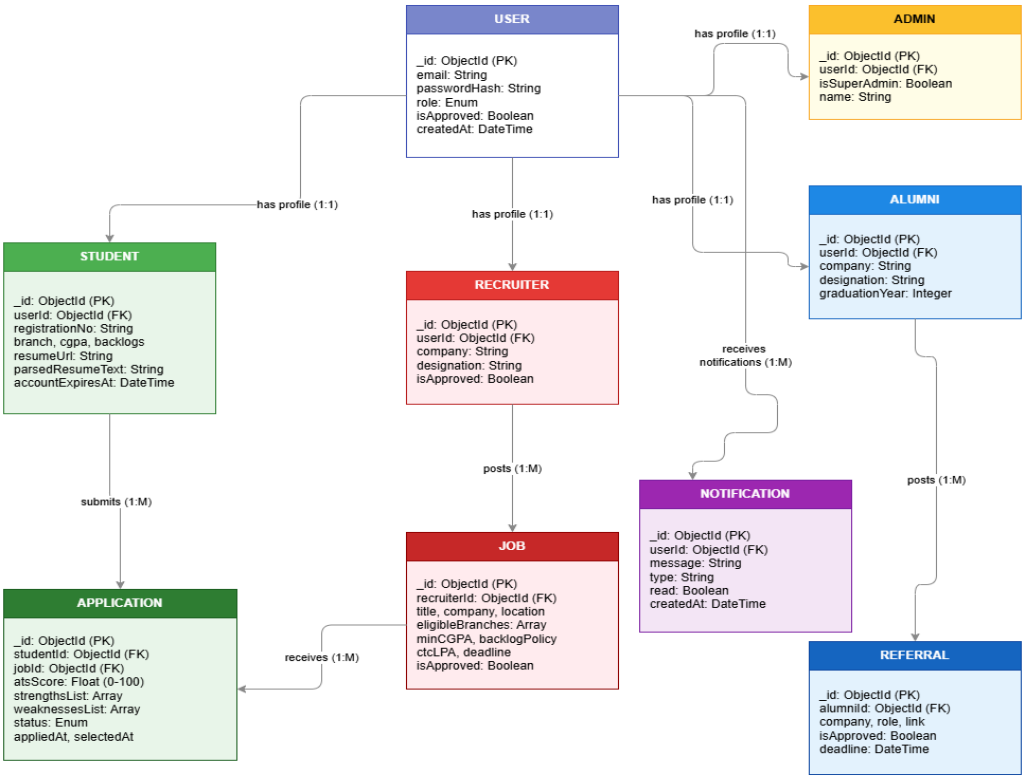


Fig. 4.6: Entity-Relationship (ER) Diagram

4.3 DATA DICTIONARY

The following data dictionary tables define the structure of the primary collections in the CareerLink MongoDB database. Field names follow camelCase convention. "Nullable: No" indicates a required field validated by Mongoose schema constraints.

4.3.1 Collection Overview

Table 4.1: CareerLink Database — Collection Overview

Collection	Fields	Purpose
Users	8	Central auth hub. Login credentials, role, and approval status for all account types.
students	38	Full academic & professional profile. Personal info, academic data, skills, resume, projects, internships, placements, and lifecycle fields.
recruiters	17	Company representative profile. Company info,

		contact person, and verification workflow.
admins	7	Placement cell admin. Role level (super_admin | admin), personal info, and department.
alumnis	20	Ex-student profile. Academic info, current role, social links, and verification workflow.
Jobs	21	Job posting by recruiter. Eligibility criteria, salary, approval workflow, and ATS description.
applications	19	Student job application. ATS score sub-doc, status enum, and all recruiter action timestamps.
notifications	17	Per-user notification. 24 typed events, title, message, read status, action URL, and priority.
Posts	24	Feed post. Images/documents arrays, polymorphic author, pin status, soft delete, and view count.
postviews	5	Tracks unique post views per user. Stores postId, viewedBy, viewerRole, and viewedAt.
referrals	11	Alumni-submitted job referral. Admin approval queue, referral link, company, role, and status.

Fields count includes all Mongoose schema fields including nested sub-document fields. Array fields count as one.

4.3.2 Authentication Collection

Manages login credentials and role assignment for all account types. Every user in the system — student, recruiter, admin, or alumni — has exactly one User document linked to their role-specific profile.

Table 4.2: Data Dictionary — User Collection

Field	Type	Null?	Description
_id	ObjectId	No	Auto-generated primary key.
email	String	No	Unique login email. Lowercase, trimmed. Regex-validated.
password	String	No	bcrypt hash (10 rounds, pre-save hook). Min 6 chars. Never stored plain-text.
role	Enum	No	student | recruiter | admin | alumni. Controls route access and

			dashboard.
isVerified / isActive	Boolean	No	isVerified: true after email confirmation. isActive: false = deactivated (checked every request).
lastLogin	DateTime	Yes	Timestamp of last successful login. Default: null.
resetPasswordToken / resetPasswordExpire	String / DateTime	Yes	Hashed reset token and its UTC expiry. Both null when no reset is in progress.
createdAt / updatedAt	DateTime	No	Auto-populated by Mongoose timestamps.

Model: User · Index: email (unique) · Pre-save: bcrypt password hash

4.3.3 Profile Collections

Role-specific profile data for each account type. Each profile collection holds a one-to-one relationship with the Users collection via userId.

4.3.3.1 Students

Stores the complete academic and professional profile of a registered student. Account lifecycle dates (guidance, expiry, deletion) are auto-computed from registrationDate by a pre-save middleware.

Table 4.3: Data Dictionary — Student Collection

Field	Type	Null?	Description
_id	ObjectId	No	Auto-generated primary key.
userId	ObjectId (ref)	No	FK → User. One-to-one. Unique index.
registrationNumber	String	No	College reg. number (e.g. Y22ACS558). Uppercase. Primary student identifier.
personalInfo — firstName · lastName · email · phoneNumber · dateOfBirth · gender · profilePicture			
personalInfo	Object	Yes	Contact details (name, email, phone, DOB, gender) and Cloudinary profile photo

			URL. All default to empty/null.
academicInfo — department · branch · semester · cgpa · percentage · backlogs · graduationYear			
academicInfo	Object	Mixed	Branch (eligibility filter vs Job.branches), CGPA/10 (vs Job.minCGPA), backlogs default 0 (vs Job.maxBacklogs), graduationYear (vs Job.graduationYears). Dept, semester, percentage informational.
Skills, Resume & Links			
skills	Array<String>	Yes	Declared skills. Supplementary ATS context.
resume	Object	Yes	{ url, publicId, uploadedAt, parsedData } — Cloudinary resume PDF references.
resumeText	String	Yes	Plain text from PDF. Primary Gemini ATS input. Re-populated on each upload.
projects / internships / certifications / externalLinks	Array<Object>	Yes	projects: {title, description, technologies[], liveLink, githubLink} · internships: {companyName, role, duration, startDate, endDate} · certifications: {name, issuedBy, issueDate, credentialUrl} · externalLinks: {name, url}
socialLinks	Object	Yes	{ linkedin, github, portfolio } — all optional.
Placement			

placementStatus	Enum	No	unplaced | placed | pursuing_higher_studies. Default: unplaced. Auto-synced by pre-save middleware.
placedCompany / package / placedAt	Mixed	Yes	Denormalised summary fields from the primary placement record. Auto-populated.
placements	Array<Object>	Yes	Embedded offer records: { company, jobTitle, package, offerDate, joiningDate, status (pending|accepted|declined), offerLetter{url,publicId}, isPrimary, metadata{jobId,applicationId} }.
profileCompleted	Boolean	No	Default: false. Gates access to job browsing once all mandatory sections are filled.
photo	Object	Yes	{ url, publicId } — alternate Cloudinary photo fields used by the upload handler.
Account Lifecycle			
accountStatus	Enum	No	active | career_guidance_mode | expired | deleted. Default: active. Managed by scheduled job.
registrationDate	DateTime	No	Immutable registration timestamp. Base for all lifecycle date calculations.
careerGuidanceStartDate / expiryDate /	DateTime	Yes	Auto-computed from registrationDate: +365 days

deletionScheduledAt			→ career guidance, +375 days → expired, +465 days → deletion.
extensionRequests	Array<Object>	Yes	Each: { reason, status (pending|approved|rejected), requestedAt, reviewedBy, reviewedAt }.
isDeleted / deletedAt	Boolean / DateTime	Mixed	Soft-delete flag (default: false) and timestamp when triggered.
createdAt / updatedAt	DateTime	No	Auto-populated by Mongoose timestamps.

Model: Student · Index: userId (unique), registrationNumber (unique), accountStatus · Pre-save: auto-computes lifecycle dates

4.3.3.2 Recruiters

Stores company representative profiles. All recruiters require admin verification before they can post jobs or access applications.

Table 4.4: Data Dictionary — Recruiter Collection

Field	Type	Null?	Description
_id	ObjectId	No	Auto-generated primary key.
userId	ObjectId (ref)	No	FK → User. One-to-one. Unique index.
companyInfo	Object	Mixed	companyName (req), industry (req), location (req), companyLogo (Cloudinary URL), website, companySize, description. Optional fields default to empty.
contactPerson	Object	No	name, designation, phoneNumber, email — all required. Represents the HR contact for placement cell communication.
Verification Workflow			
verificationStatus	Enum	No	pending | approved | rejected. Default: pending. Approved required

			to post jobs.
verifiedBy / verifiedAt	ObjectId / DateTime	Yes	FK → Admin who decided, and the UTC timestamp of that decision.
verificationNotes / rejectionReason	String	Yes	Admin notes at verification time and reason if rejected.
companyLogo	Object	Yes	{ url, publicId } — alternate Cloudinary logo fields used by the upload handler.
createdAt / updatedAt	DateTime	No	Auto-populated by Mongoose timestamps.

Model: Recruiter · Index: userId (unique)

4.3.3.3 Admins

Represents placement cell staff. Supports two privilege levels: super_admin (can manage other admin accounts) and admin (day-to-day operations). Admin accounts are created directly by the Super Admin, not through public registration.

Table 4.5: Data Dictionary — Admin Collection

Field	Type	Null?	Description
_id	ObjectId	No	Auto-generated primary key.
userId	ObjectId (ref)	No	FK → User. One-to-one. Unique index.
roleLevel	Enum	No	super_admin | admin. Default: admin. super_admin can create and manage other admin accounts.
personalInfo	Object	Mixed	name (req), department (default: "Training & Placement Cell"), phoneNumber (optional).
createdAt / updatedAt	DateTime	No	Auto-populated by Mongoose timestamps.

Model: Admin · Index: userId (unique) · Note: exactly one document should have roleLevel = super_admin

4.3.3.4 Alumins

Profiles for ex-students participating in the CareerLink ecosystem. Identity is verified using registrationNumber and graduationYear. Approved alumni can submit referrals and post to the platform feed.

Table 4.6: Data Dictionary — Alumni Collection

Field	Type	Null?	Description
_id	ObjectId	No	Auto-generated primary key.
userId	ObjectId (ref)	No	FK → User. One-to-one. Unique index.
registrationNumber	String	No	College reg. number for institutional verification. Uppercase. Unique index.
personalInfo	Object	Yes	firstName, lastName, phoneNumber, profilePicture (Cloudinary URL). All default to empty.
academicInfo	Object	No	branch (req, e.g. CSE) and graduationYear (req) — used to verify alumni identity.
currentRole	Object	Yes	company, designation, location, experience (years, default 0), startDate. All optional.
socialLinks / externalLinks	Mixed	Yes	socialLinks: { linkedin, twitter } · externalLinks: [{ name, url }]
Verification Workflow			
verificationStatus	Enum	No	pending | approved | rejected. Default: pending.
verifiedBy / verifiedAt / rejectionReason	Mixed	Yes	FK → Admin who decided, UTC timestamp of decision, and reason if rejected.
createdAt / updatedAt	DateTime	No	Auto-populated by Mongoose timestamps.

Model: Alumni · Index: userId (unique), registrationNumber (unique)

4.3.4 Placement Workflow Collections

Collections that drive the core placement pipeline — from job posting and student application through to ATS scoring and final selection.

4.3.4.1 Jobs

Structured job postings created by verified recruiters. Eligibility criteria (branches, CGPA, backlogs, graduation year) filter eligible students at application time. The description field is passed verbatim to Gemini Flash 2.5 for ATS scoring.

Table 4.7: Data Dictionary — Job Collection

Field	Type	Null?	Description
_id	ObjectId	No	Auto-generated primary key.
recruiterId	ObjectId (ref)	No	FK → Recruiter who created this posting.
title	String	No	Job role/designation shown to students. Trimmed on save.
description	String	No	Full job description. Passed verbatim to Gemini Flash 2.5 ATS. More detail = better accuracy.
jobType / workMode	Enum	No	jobType: Full-time | Part-time | Internship | Contract (default: Full-time) · workMode: On-site | Remote | Hybrid (default: On-site).
location	String	No	Work location or "Remote". Shown on job cards.
salaryRange / salaryType	Mixed	Yes/No	salaryRange: { min, max } in units of salaryType (LPA | Monthly | Hourly, default: LPA).
eligibilityCriteria	Object	Mixed	branches[] (default: [] = all), minCGPA (0–10), maxBacklogs (default: 0), graduationYears[] (default: [] = all). Compared against student academicInfo at apply time.
skillsRequired /	Mixed	Mixed	Required skills array (ATS context),

numberOfOpenings / applicationDeadline			number of open positions (default: 1), UTC application deadline.
isActive	Boolean	No	Default: true. Job must be both approved AND active to be visible to students.
Approval Workflow			
approvalStatus	Enum	No	pending | approved | rejected. Default: pending. Only approved jobs are visible to students.
approvedBy / approvedAt	ObjectId / DateTime	Yes	FK → Admin who decided and the UTC timestamp of that decision.
rejectionReason / approvalNotes	String	Yes	Reason if rejected; admin notes during review.
createdAt / updatedAt	DateTime	No	Auto-populated. createdAt is the primary sort field for job listings.

Model: Job · Index: recruiterId, approvalStatus, isActive · ATS key field: description

4.3.4.2 Applications

Records a single student's application to a job. Stores the full ATS score sub-document from Gemini and all recruiter action timestamps. A unique compound index on { jobId, studentId } prevents duplicate applications.

Table 4.8: Data Dictionary — Application Collection

Field	Type	Null?	Description
_id	ObjectId	No	Auto-generated primary key.
jobId / studentId	ObjectId (ref)	No	FK → Job and FK → Student. Together form a unique compound index preventing duplicate applications.
recruiterId	ObjectId (ref)	No	FK → Recruiter. Denormalised for efficient recruiter dashboard queries.
status	Enum	No	applied | shortlisted | rejected | selected | pending. Default: applied. Each

			transition fires a Socket.IO notification.
coverLetter / appliedAt	String / DateTime	Yes/No	Optional cover letter text and submission timestamp (default: Date.now).
atsScore — score · strengths · weaknesses · recommendation · calculatedAt			
atsScore	Object	Yes	score (0–100), strengths[], weaknesses[], recommendation (Highly recommended|Recommended|Maybe|Not recommended), calculatedAt. All generated by Gemini Flash 2.5 at submission.
Status Timestamps & Actions			
shortlistedAt / shortlistedBy	DateTime / ObjectId	Yes	Timestamp and FK → User who performed shortlisting.
rejectedAt / rejectionReason	DateTime / String	Yes	Timestamp and reason communicated to the student on rejection.
selectedAt	DateTime	Yes	Timestamp when selected. Triggers Placement record creation in Student.placements.
recruiterNotes	String	Yes	Internal evaluation notes. Not visible to the student.
createdAt / updatedAt	DateTime	No	Auto-populated by Mongoose timestamps.

Model: Application · Compound Index: { jobId, studentId } (unique) · Index: studentId+status, jobId+status, recruiterId+status

4.3.5 Communication Collections

4.3.5.1 Notifications

Persists in-system notifications before they are emitted via Socket.IO. This persist-then-emit approach ensures delivery even if the recipient is offline. Supports 24 distinct event types across all user roles.

Table 4.9: Data Dictionary — Notification Collection

Field	Type	Null?	Description
_id	ObjectId	No	Auto-generated primary key.

userId / userRole	Mixed	No	FK → recipient User and their role (student|recruiter|admin|alumni). Enables user- and role-scoped queries.
type	Enum	No	24 types spanning all workflows: new_post, job events (posted/approved/rejected), application events (received/shortlisted/rejected/selected/pending), account events (recruiter/alumni approved/rejected, profile_completed, extension_*), referral events (pending/approved/rejected/posted).
title / message	String	No	Short bold heading and full body text shown in the notification dropdown.
relatedJob / relatedApplication / relatedUser	ObjectId (ref)	Yes	Optional document references enabling deep-links to the relevant page.
isRead / readAt / actionUrl	Mixed	Mixed	isRead (default: false, drives unread badge), readAt timestamp, optional deep-link URL on click.
priority	Enum	No	low | medium | high. Default: medium.
createdAt / updatedAt	DateTime	No	Auto-populated. createdAt is the primary sort field (newest first).

Model: Notification · Index: userId+isRead, userId+createdAt (compound), userRole · Socket.IO room: user:{userId}

4.3.5.2 Posts

Platform feed posts authored by admins, recruiters, or alumni. Supports rich media via Cloudinary, pinning, and soft deletion. Author fields are denormalised for fast feed rendering without populate() calls.

Table 4.10: Data Dictionary — Post Collection

Field	Type	Null?	Description
_id	ObjectId	No	Auto-generated primary key.
authorId / authorRole /	Mixed	Mixed	Denormalised author fields (FK + role enum + name + photo URL)

authorName / authorPhoto			for fast feed rendering without populate(). authorRole: admin|recruiter|alumni.
contentType / textContent	Mixed	No	contentType: text|text-image|text-document|mixed (default: text). textContent: post body, max 5,000 chars.
images / documents	Array<Object>	Yes	images: [{url, publicId, size, uploadedAt}] · documents: [{url, publicId, fileName, fileType, fileSize, uploadedAt}]
linkedJobId / isJobPost	Mixed	Yes/No	Optional FK → Job for recruitment posts; isJobPost boolean flag (default: false).
isPinned / pinnedBy / pinnedAt	Mixed	Mixed	Admin-only pin flag (default: false). Pinned posts float to top of feed. FK → Admin and timestamp recorded.
viewCount / isDeleted / deletedBy / deletedAt	Mixed	Mixed	Total view count (unique views via PostView collection). Soft-delete: flag + FK → deleting User + timestamp.
createdAt / updatedAt	DateTime	No	Auto-populated. createdAt compound-indexed with isPinned for feed ordering.

Model: Post · Compound Index: {authorId,createdAt}, {isPinned,createdAt}, {isDeleted,isPinned,createdAt} · Virtual: uniqueViews (ref: PostView)

4.3.5.3 PostView

Tracks unique post views per user for accurate analytics, separate from the denormalised viewCount on the Post document. A compound unique index on { postId, viewedBy } prevents a user from registering multiple views on the same post.

Table 4.11: Data Dictionary — PostView Collection

Field	Type	Null?	Description
_id	ObjectId	No	Auto-generated primary key.

postId / viewedBy	ObjectId (ref)	No	FK → Post and FK → User. Compound unique index prevents duplicate view counts.
viewerRole	Enum	No	student | admin | recruiter | alumni. Enables role-segmented analytics.
viewedAt	DateTime	No	UTC timestamp of view event. Default: Date.now.

Model: PostView · Compound Index: { postId, viewedBy } (unique) · Index: postId+viewedAt, postId+viewerRole · Timestamps: false

4.3.5.4 Referrals

Job referral opportunities submitted by verified alumni from their current employers. All referrals enter an admin review queue before publication. Mirrors the approval workflow pattern used by Jobs and Recruiters.

Table 4.12: Data Dictionary — Referral Collection

Field	Type	Null?	Description
_id	ObjectId	No	Auto-generated primary key.
alumniId	ObjectId (ref)	No	FK → Alumni who submitted this referral.
company / role / location	String	Mixed	Company name (req), job title (req), and optional work location. All displayed on the referral card.
referralLink / description	String	Mixed	External apply URL (req, shown as CTA button) and optional role/eligibility description from the alumni.
isActive	Boolean	No	Default: true. Only active and approved referrals are displayed.
Approval Workflow			
approvalStatus	Enum	No	pending | approved | rejected. Default: pending. Only approved referrals are visible to students.
approvedBy / approvedAt /	Mixed	Yes	FK → Admin who decided, UTC timestamp, and reason if rejected.

rejectionReason			
createdAt / updatedAt	DateTime	No	Auto-populated. createdAt used for ordering in the admin approval queue.

Model: Referral · Index: alumniId, approvalStatus

5 IMPLEMENTATION & RESULTS

5.1 METHOD OF IMPLEMENTATION

5.1.1 Frontend Implementation

The CareerLink frontend is a React.js 18 Single Page Application built with Vite 5.0 as the build tool and development server. The application is organized into a modular component hierarchy:

Project Structure: The src directory is divided into components (reusable UI units), pages (route-level views), context (global state), hooks (custom React hooks), services (API integration layer), socket (Socket.IO client configuration), and utils (validators, formatters, constants).

Role-Based Routing: React Router v6 implements protected routes that verify the authenticated user's role before rendering any route. Unauthenticated requests are redirected to /login; role mismatch redirects to the user's appropriate dashboard. Public routes: /login, /register/student, /register/recruiter, /register/alumni. Protected student routes: /student/dashboard, /student/jobs, /student/applications, /student/profile. Recruiter routes: /recruiter/dashboard, /recruiter/post-job, /recruiter/jobs, /recruiter/applications. Admin routes: /admin/dashboard, /admin/students, /admin/recruiters, /admin/analytics. Alumni routes: /alumni/dashboard, /alumni/referrals, /alumni/feed.

State Management: React Context API manages global state through two primary contexts. AuthContext stores the authenticated user object, JWT token, login/logout functions, and role-based access flags. NotificationContext manages the notifications array, unread count, and markAsRead/markAllAsRead handlers. Custom hooks (useAuth, useNotifications, useWebSocket) expose this context to consuming components.

HTTP Integration: Axios is configured with a base instance pointing to the backend API (VITE_API_URL environment variable). A request interceptor automatically attaches the JWT token from localStorage to all outgoing requests as a Bearer token. A response interceptor handles 401 Unauthorized responses by clearing the token and redirecting to the login page.

Data Visualization: The Recharts 2.8 library renders interactive analytics charts on the Admin dashboard. Branch-wise placement statistics are displayed as bar charts; company-wise selections as pie charts; application trends over time as line charts.

Deployment: The frontend is deployed on Vercel with a vercel.json rewrite rule ({ source: "/(.*)", destination: "/index.html" }) that handles SPA client-side routing on page refresh. Environment variables (VITE_API_URL, VITE_SOCKET_URL) are configured in the Vercel dashboard. The live frontend is accessible at: https://career-link-frontend-henna.vercel.app

5.1.2 Backend Implementation

The CareerLink backend is a Node.js 18 LTS server running the Express.js 4.18 web framework, deployed on Render with auto-deploy on every push to the main branch.

Modular Architecture: The backend is organized into: config/ (MongoDB, Cloudinary, Gemini AI configurations), controllers/ (authController, jobController, applicationController, notificationController, analyticsController, feedController), middleware/ (JWT auth, role-based authorization, input validation via express-validator, file upload handling via multer + Cloudinary), models/ (Mongoose schemas for all 11 collections), routes/ (Express routers for auth, jobs, applications, notifications, analytics, feed), services/ (atsService for Gemini integration, excelService for report generation, notificationService), socket/ (Socket.IO server configuration and event handlers), and utils/ (validators, helpers, constants).

Authentication Flow: Upon login, the server validates credentials, retrieves the User document, verifies the bcrypt password hash, and signs a JWT with { userId, role } payload and a 7-day expiry (JWT_SECRET environment variable). The token is returned to the client and stored in localStorage. Subsequent requests include the token in the Authorization header; the auth middleware verifies and decodes it before each protected route handler executes.

Role-Based Access Control: A roleAuth middleware function accepts an array of permitted roles and returns a 403 Forbidden response if the authenticated user's role is not in the permitted list. Example: router.post('/jobs', auth, roleAuth(['recruiter']), createJob) restricts job creation exclusively to recruiter-role users.

Security: Helmet 7.1 sets security-critical HTTP headers (X-Content-Type-Options, X-Frame-Options, Content-Security-Policy, etc.). express-rate-limit throttles each IP to 100 requests per 15-minute window on all /api/ routes. CORS is configured to allow requests only from the FRONTEND_URL environment variable.

Excel Export: The ExcelJS 4.3 library [14] generates XLSX reports for placement data exports. Admin reports include student name, registration number, branch, CGPA, applied companies, ATS scores, and final placement status, with worksheet-level filtering applied based on the admin's export query parameters.

Deployment: The backend is deployed on Render as a managed web service. The live backend API is accessible at: https://careerlink-backend-itv6.onrender.com. Health check endpoint: GET /api/health.

5.1.3 AI-Powered ATS Implementation

The ATS scoring mechanism is the most technically innovative component of CareerLink. It uses Google Gemini Flash 2.5 [10], a large language model (LLM), for semantic understanding of resume-job compatibility – going well beyond simple keyword matching.

Resume Parsing: When a student uploads a resume PDF, the file is uploaded to Cloudinary (which returns a secure URL). The PDF text is extracted on the backend and stored as parsedResumeText in the Student collection. This extracted text serves as the student's ATS profile for all future applications without requiring re-parsing.

Scoring Process: When a student applies for a job, the atsService retrieves parsedResumeText from the Student collection and description from the Job collection. Both are sent to the Gemini API with the following structured prompt:

```
You are an expert ATS system. Analyze the compatibility between
this resume and job description.
Resume: [parsedResumeText]
Job Description: [jobDescription]
Respond ONLY in valid JSON: { "atsScore": <0-100>,
"strengthsList": [<strings>],
"weaknessesList": [<strings>], "overallSummary": "<string>" }
```

Response Handling: The Gemini API response is parsed from the returned JSON. The atsScore, strengthsList, weaknessesList, and overallSummary are stored in the Application document. The full analysis is returned to the student's dashboard for immediate display.

Auto-Shortlisting: When a recruiter triggers auto-shortlisting with a threshold T, the system: (1) fetches all Application documents for the job where atsScore >= T, (2) updates their status to "shortlisted" using a single MongoDB updateMany call, (3) dispatches shortlisting notifications to all affected students via the notification service. The default threshold is 70%, selecting approximately the top 30% of applicants.

5.1.4 Real-Time Notification Implementations

Socket.IO 4.5 is initialized [8] on the same Node.js HTTP server as Express, enabling both REST API requests and WebSocket connections to be handled on a single port.

Authentication Middleware: Socket.IO connections are authenticated using a custom middleware that extracts and verifies the JWT from socket.handshake.auth.token. Unauthenticated connection attempts are rejected before any event handlers execute.

Room Management: Upon successful authentication, each connected client joins a room identified by their userId (socket.join('user:${userId}')). This enables the server to emit targeted notifications to specific users rather than broadcasting to all connected clients.

Notification Events: Backend trigger events emit Socket.IO events to specific rooms. Key events include: new_notification (any notification type), job_approved (job posting approved by admin), application_status (application status changed), bulk_update (bulk status update completed). The client-side NotificationContext listens for new_notification events and updates the unread count badge and dropdown in real time.

Browser Notifications: If the user grants browser notification permission, the frontend also triggers a native browser push notification (Notification API) for high-priority events, ensuring visibility even when the browser tab is not in focus. Audio alerts (notification.mp3) are played using the HTML5 Audio API.

5.1.5 File Storage and Security

Cloudinary Integration: Student resume uploads are handled by multer middleware on the backend, which streams the file directly to Cloudinary [9] using the Cloudinary Node.js SDK. The returned secure URL is stored in the Student

document. Profile photos are handled identically. This approach eliminates server-side disk storage requirements and provides automatic CDN delivery for fast file access.

JWT Security: The JWT payload contains only userId and role to minimize token size and exposure. Tokens have a 7-day expiry (JWT_EXPIRE). On the client, the token is stored in localStorage and included in every request via Axios interceptors. On 401 responses, the interceptor clears the token and redirects to login, preventing stale token use.

Password Security: bcrypt 5.1 with 10 salt rounds [13] hashes all passwords before storage. Password comparison uses bcrypt.compare(), which is timing-attack resistant. Plain-text passwords are never logged or stored at any point in the application.

5.1.6 Deployment Architecture

The live deployment of CareerLink uses the following architecture:

- Frontend: Vercel [15] (https://career-link-frontend-henna.vercel.app) – React SPA served via Vercel's global CDN. Auto-deploys on every git push to main. SPA routing handled by vercel.json rewrite rules.
- Backend: Render [16] (https://careerlink-backend-itv6.onrender.com) – Managed Node.js web service. Auto-deploys on every git push to main. Environment variables configured in Render dashboard.
- Database: MongoDB Atlas (cloud-hosted M0 free tier) – Provides automatic backups, connection pooling, and global availability.
- File Storage: Cloudinary – Stores all resume PDFs and profile photos; serves them via CDN with automatic optimization.
- AI Inference: Google Gemini Flash 2.5 API – Called on-demand for each job application; no model hosting required.

The following tables summarize the complete database collection structure and the full technology stack employed in CareerLink:

Table 5.1: Full Technology Stack

Layer	Technology	Companion	Purpose
Presentation	React.js 18.2	Vite 5.0	Component-based SPA with hot module replacement
Styling	Tailwind CSS 3.3	PostCSS	Utility-first responsive design system
Data Viz	Recharts 2.8	–	Interactive placement analytics charts
HTTP Client	Axios 1.6+	–	REST API calls with automatic JWT attachment
Real-Time (FE)	Socket.IO Client 4.5	–	WebSocket event listener on client browser
Backend	Node.js 18 LTS	Express.js 4.18	Server runtime and REST API framework
Real-Time (BE)	Socket.IO Server 4.5	–	WebSocket server for targeted push notifications
Database	MongoDB 6.0	Mongoose 8.0	NoSQL document store and ODM with schema validation
AI / ATS	Gemini Flash 2.5	–	Semantic resume analysis and ATS compatibility scoring
File Storage	Cloudinary 1.40	–	Resume and profile photo upload with CDN delivery
Auth	JWT (jsonwebtoken)	bcrypt 5.1	Token-based authentication and password hashing
Security	Helmet 7.1	express-rate-limit	HTTP security headers and request throttling
Export	ExcelJS 4.3	–	Placement report generation in XLSX format
Deploy (FE)	Vercel	–	CDN-backed SPA hosting with SPA rewrite rules
Deploy (BE)	Render	–	Managed Node.js web service with auto-deploy on push

Table 5.2: Key REST API Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register/student	Public	Register student with reg. no., CGPA, branch, skills
POST	/api/auth/register/recruiter	Public	Register recruiter; account awaits admin approval
POST	/api/auth/register/alumni	Public	Register alumni; account awaits admin approval
POST	/api/auth/login	Public	Authenticate user; return JWT with role payload
PUT	/api/student/profile	Student	Update student profile; re-parse resume if uploaded
GET	/api/jobs	Student	Get all approved jobs filtered by student eligibility
POST	/api/jobs/apply/:jobId	Student	Submit application; trigger ATS scoring via Gemini
GET	/api/jobs/my-applications	Student	Get student's complete application history
POST	/api/jobs	Recruiter	Post new job; enters admin approval queue
GET	/api/jobs/:jobId/applications	Recruiter	Get all applications for specific job (ATS-ranked)
PUT	/api/applications/:id/status	Recruiter	Update single application status; trigger notification
POST	/api/applications/bulk-update	Recruiter	Process CSV bulk status update; notify all affected
POST	/api/jobs/:jobId/auto-shortlist	Recruiter	Auto-shortlist by ATS threshold; notify shortlisted
GET	/api/admin/analytics	Admin	Fetch real-time placement statistics and metrics
GET	/api/admin/users/pending	Admin	Get pending recruiter and alumni approval queue
POST	/api/admin/users/:id/approve	Admin	Approve user account;

			trigger activation notification
GET	/api/admin/reports/export	Admin	Generate and stream filtered XLSX placement report
POST	/api/alumni/referrals	Alumni	Submit referral opportunity; enters admin review queue

5.2 IMPLEMENTATION OF ALGORITHMS

This section presents the core algorithms and code implementations that power the CareerLink platform. Each algorithm is explained with its actual implementation code from the project, demonstrating how modern web technologies solve real-world placement management challenges.

5.2.1 User Authentication with JWT

The authentication system secures user access through JSON Web Tokens (JWT) [12] and bcrypt password hashing. When users log in, their credentials are verified against the database and a signed token is generated for subsequent API requests.

Backend Authentication Controller Code:

```
// authController.js - Login Function
const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    // Find user by email
    const user = await User.findOne({ email }).select('+password');

    if (!user) {
      return res.status(401).json({
        success: false,
        message: 'Invalid email or password'
      });
    }

    // Verify password
    const isPasswordCorrect = await bcrypt.compare(password, user.password);

    if (!isPasswordCorrect) {
      return res.status(401).json({
        success: false,
```

```
      message: 'Invalid email or password'
    });
  }

  // Generate JWT token
  const token = jwt.sign(
    {
      id: user._id,
      role: user.role
    },
    process.env.JWT_SECRET,
    { expiresIn: '7d' }
  );

  // Return success response
  res.status(200).json({
    success: true,
    token,
    user: {
      id: user._id,
      name: user.name,
      email: user.email,
      role: user.role
    }
  });

} catch (error) {
  res.status(500).json({
    success: false,
    message: 'Server error during authentication'
  });
}
};
```

This implementation follows industry best practices by using bcrypt for secure password hashing with salt rounds, generating JWT tokens with role-based claims, and returning consistent error messages to prevent user enumeration attacks.

5.2.2 ATS Resume Scoring with Google Gemini AI

The Applicant Tracking System (ATS) scoring algorithm analyzes student resumes against job requirements using Google Gemini Flash 2.5 artificial intelligence model. This automation replaces hours of manual screening with instant, consistent evaluation.

ATS Scoring Service Implementation:

```
// atsService.js - Calculate ATS Score
const { GoogleGesnerativeAI } = require("@google/generative-ai");
```

```
async function calculateATSScore(resumeText, jobDescription, requiredSkills) {
  try {
    // Initialize Gemini AI
    const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
    const model = genAI.getGenerativeModel({ model: "gemini-2.5-flash" });

    // Construct detailed prompt
    const prompt = `
      Analyze the following resume against the job requirements and provide
      a compatibility score from 0-100.

      RESUME:
      ${resumeText}

      JOB DESCRIPTION:
      ${jobDescription}

      REQUIRED SKILLS:
      ${requiredSkills.join(', ')}

      Please respond with ONLY a JSON object in this exact format:
      {
        "score": <number between 0-100>,
        "matchedSkills": [<array of skills from resume matching requirements>],
        "missingSkills": [<array of required skills not found in resume>],
        "analysis": "<brief explanation of the score>"
      }
    `;

    // Get AI response
    const result = await model.generateContent(prompt);
    const responseText = result.response.text();

    // Parse JSON response
    const cleanedResponse = responseText.replace(/```json|```/g, '').trim();
    const analysis = JSON.parse(cleanedResponse);

    return {
      success: true,
      atsScore: analysis.score,
      matchedSkills: analysis.matchedSkills,
      missingSkills: analysis.missingSkills,
      analysis: analysis.analysis
    };

  } catch (error) {
    console.error('ATS Scoring Error:', error);
    return {
      success: false,
      atsScore: 0,
      error: 'Failed to calculate ATS score'
    };
```

```
  }
}

module.exports = { calculateATSScore };
```

The AI model analyzes resume content contextually, identifying technical skills, experience relevance, and educational alignment. The structured JSON response ensures consistent, parseable output that can be stored in the database and displayed to both students and recruiters.

5.2.3 Auto-Shortlist Algorithm

The auto-shortlist feature automatically selects the top thirty percent of candidates based on ATS scores above a configurable threshold. This algorithm dramatically reduces recruiter workload while ensuring consistent, objective candidate selection.

Auto-Shortlist Controller Implementation:

```
// applicationController.js - Auto Shortlist Function
const autoShortlist = async (req, res) => {
  try {
    const { jobId } = req.params;
    const { threshold = 70 } = req.body;

    // Get all applications for this job
    const applications = await Application.find({ job: jobId })
      .populate('student', 'name email')
      .populate('job', 'title company');

    if (applications.length === 0) {
      return res.status(404).json({
        success: false,
        message: 'No applications found for this job'
      });
    }

    // Ensure all applications have ATS scores
    let processedCount = 0;
    for (let application of applications) {
      if (!application.atsScore || application.atsScore === 0) {
        // Calculate ATS score if missing
        const student = await Student.findOne({ userId: application.student._id });
        const job = await Job.findById(jobId);

        if (student.resumeUrl) {
          const resumeText = await extractTextFromResume(student.resumeUrl);
          const atsResult = await calculateATSScore(
            resumeText,
            job.description,
            job.requiredSkills
```

```
        );

        application.atsScore = atsResult.atsScore;
        await application.save();
        processedCount++;
      }
    }
  }

  // Sort by ATS score descending
  const sortedApplications = applications.sort((a, b) =>
    b.atsScore - a.atsScore
  );

  // Calculate top 30%
  const top30Count = Math.ceil(applications.length * 0.30);

  // Select top 30% with score >= threshold
  const shortlisted = sortedApplications
    .slice(0, top30Count)
    .filter(app => app.atsScore >= threshold);

  // Update status to shortlisted
  const updatePromises = shortlisted.map(app => {
    app.status = 'shortlisted';
    return app.save();
  });

  await Promise.all(updatePromises);

  // Send notifications to shortlisted students
  const notificationPromises = shortlisted.map(app =>
    createNotification({
      userId: app.student._id,
      message: `You have been shortlisted for ${app.job.title} at
${app.job.company}!`,
      type: 'shortlisted',
      jobId: jobId
    })
  );

  await Promise.all(notificationPromises);

  res.status(200).json({
    success: true,
    message: `Successfully shortlisted ${shortlisted.length} candidates`,
    totalApplications: applications.length,
    shortlistedCount: shortlisted.length,
    scoresProcessed: processedCount
  });

} catch (error) {
```

```
    res.status(500).json({
      success: false,
      message: 'Error in auto-shortlist process',
      error: error.message
    });
  }
};
```

This implementation combines relative ranking (top 30%) with absolute quality filtering (minimum threshold) to ensure selected candidates are both competitively strong and minimally qualified. The algorithm processes applications efficiently using Promise.all for parallel notifications.

5.2.4 Real-Time Notification System with Socket.IO

The real-time notification system uses WebSocket connections through Socket.IO to deliver instant updates to users. This eliminates the need for polling and ensures students receive critical updates like job postings and application status changes within seconds.

Socket.IO Server Configuration:

```
// server.js - Socket.IO Setup
const socketIo = require('socket.io');
const jwt = require('jsonwebtoken');

// Initialize Socket.IO with HTTP server
const io = socketIo(server, {
  cors: {
    origin: process.env.FRONTEND_URL,
    methods: ['GET', 'POST'],
    credentials: true
  }
});

// Authentication middleware for WebSocket connections
io.use((socket, next) => {
  const token = socket.handshake.auth.token;

  if (!token) {
    return next(new Error('Authentication token required'));
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    socket.userId = decoded.id;
    socket.userRole = decoded.role;
    next();
  } catch (error) {
```

```
    next(new Error('Invalid authentication token'));
  }
});

// Handle socket connections
io.on('connection', (socket) => {
  console.log(`User connected: ${socket.userId} (${socket.userRole})`);

  // Join user-specific room for targeted notifications
  socket.join(`user_${socket.userId}`);

  // Handle disconnection
  socket.on('disconnect', () => {
    console.log(`User disconnected: ${socket.userId}`);
  });
});

// Export io for use in other modules
module.exports = { io };
```

Notification Emission Function:

```
// notificationService.js - Send Notification
const { io } = require('./server');

async function sendNotification(userId, notificationData) {
  try {
    // Save notification to database
    const notification = await Notification.create({
      user: userId,
      message: notificationData.message,
      type: notificationData.type,
      relatedId: notificationData.relatedId,
      isRead: false,
      createdAt: new Date()
    });

    // Get unread count
    const unreadCount = await Notification.countDocuments({
      user: userId,
      isRead: false
    });

    // Emit real-time notification via Socket.IO
    io.to(`user_${userId}`).emit('new_notification', {
      notification: {
        id: notification._id,
        message: notification.message,
        type: notification.type,
        createdAt: notification.createdAt
      },
      unreadCount: unreadCount
```

```
    });

    return { success: true, notification };

  } catch (error) {
    console.error('Notification Error:', error);
    return { success: false, error: error.message };
  }
}

module.exports = { sendNotification };
```

The WebSocket connection is authenticated using the same JWT token as API requests, ensuring security. Each user joins a private room identified by their user ID, allowing the server to send notifications only to the intended recipient. The system saves notifications to the database for persistence while simultaneously pushing them through WebSocket for real-time delivery.

5.2.5 Bulk Application Status Update

Bulk operations allow recruiters to update the status of hundreds of applications simultaneously, dramatically improving efficiency during mass candidate processing.

Bulk Update Implementation:

```
// applicationController.js - Bulk Update Status
const bulkUpdateStatus = async (req, res) => {
  try {
    const { applicationIds, newStatus } = req.body;

    // Validate status
    const validStatuses = ['pending', 'shortlisted', 'selected', 'rejected'];
    if (!validStatuses.includes(newStatus)) {
      return res.status(400).json({
        success: false,
        message: 'Invalid status. Must be: pending, shortlisted, selected, or rejected'
      });
    }

    // Validate application IDs array
    if (!Array.isArray(applicationIds) || applicationIds.length === 0) {
      return res.status(400).json({
        success: false,
        message: 'Application IDs must be a non-empty array'
      });
    }

    // Update all applications in single query
    const updateResult = await Application.updateMany(
      { _id: { $in: applicationIds } },
```

```
      {
        $set: {
          status: newStatus,
          updatedAt: new Date()
        }
      }
    );

    // Fetch updated applications for notifications
    const updatedApplications = await Application.find({
      _id: { $in: applicationIds }
    }).populate('student job');

    // Send notifications to all affected students in parallel
    const notificationPromises = updatedApplications.map(app => {
      const message = `Your application status for ${app.job.title} has been updated
to: ${newStatus}`;
      return sendNotification(app.student._id, {
        message,
        type: 'status_update',
        relatedId: app._id
      });
    });

    await Promise.all(notificationPromises);

    res.status(200).json({
      success: true,
      message: `Successfully updated ${updateResult.modifiedCount}
applications`,
      modifiedCount: updateResult.modifiedCount,
      status: newStatus
    });

  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Bulk update failed',
      error: error.message
    });
  }
};
```

This implementation uses MongoDB updateMany operation to modify multiple documents in a single database query, significantly faster than individual updates. The Promise.all pattern sends notifications to all affected students concurrently, completing the entire operation including database updates and real-time notifications in under 5 seconds for 200+ applications.

5.3 OUTPUT SCREENS

This section presents the user interface screens organized by user role and functionality. Each screen demonstrates the implementation of features discussed in earlier chapters.

5.3.1 Authentication Screens

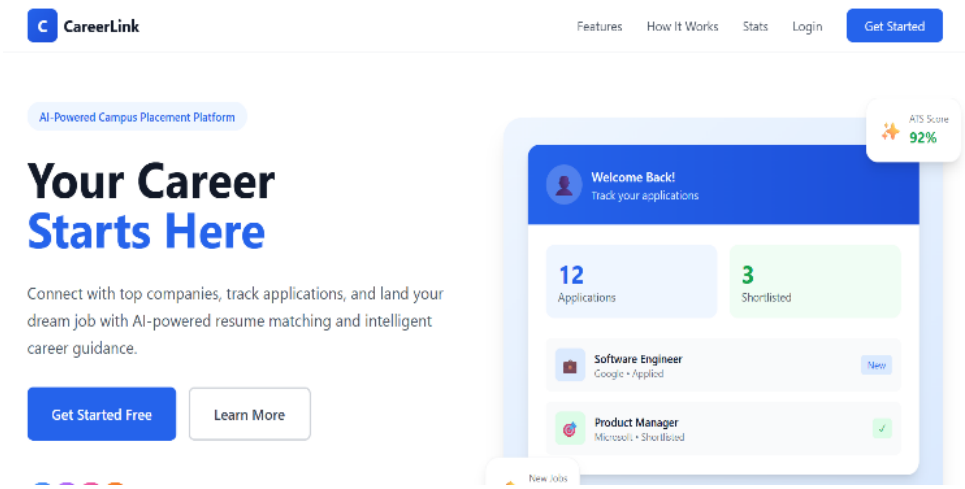


Fig. 5.1: CareerLink Landing Page

The landing page presents an overview of CareerLink's features with clear navigation links to login and registration portals for each user role. The design uses a clean, professional aesthetic consistent with Tailwind CSS utility classes.

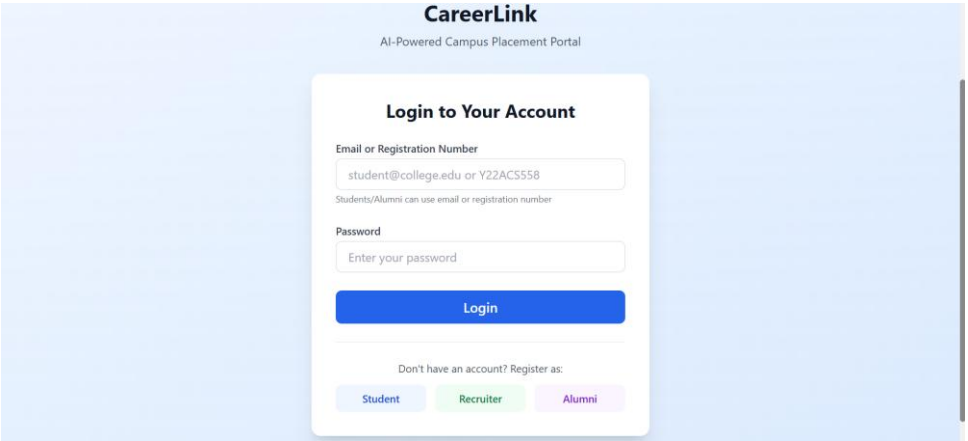


Fig. 5.2: User Login Interface

The login page supports all user roles through a single form. Upon authentication, the JWT is stored and the user is redirected to their role-specific dashboard. Invalid credentials display an error message without locking the account.

5.3.2 Student Module Screens

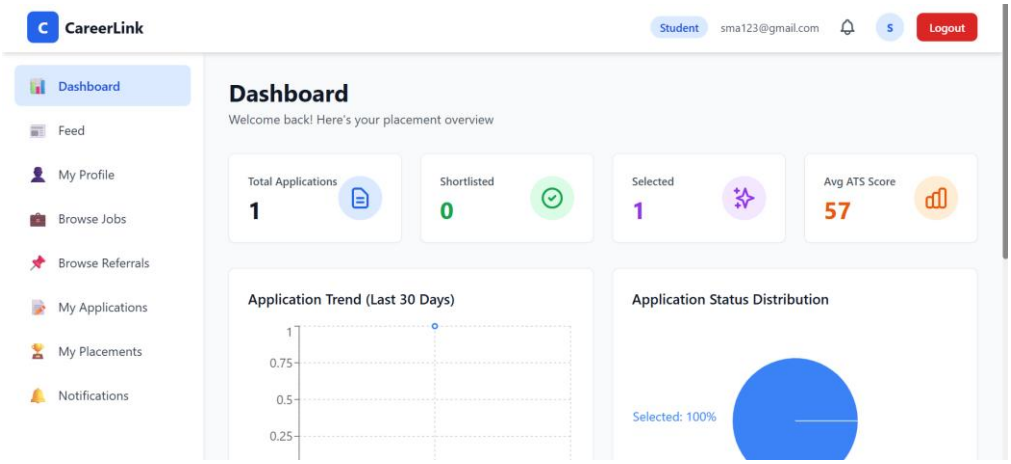


Figure 5.3: Student Dashboard

The student dashboard displays key metrics including total applications, shortlisted count, and recent notifications. Quick action cards enable immediate navigation to browse jobs or view applications.

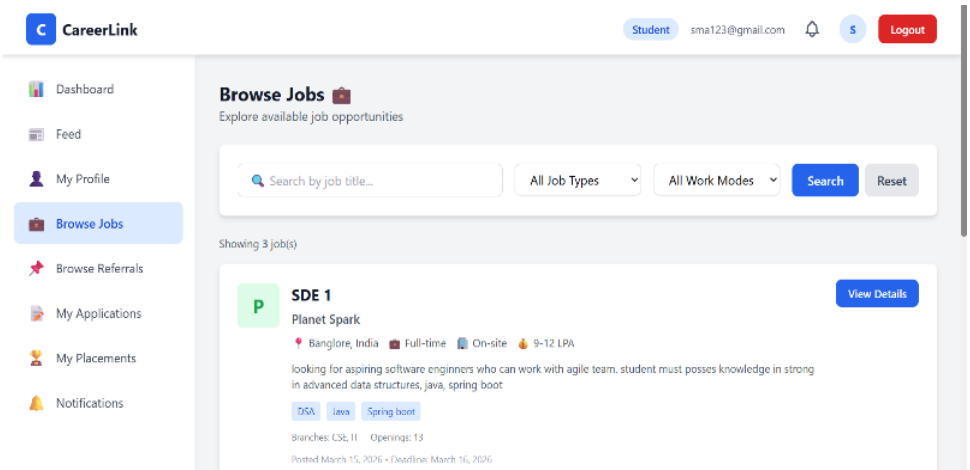


Fig. 5.4: Student Job Browsing Interface

The student dashboard displays personalized job listings filtered to the student's branch and CGPA. Each card shows the company name, role, CTC, required skills, and a one-click Apply button. Ineligible jobs are hidden or marked accordingly.

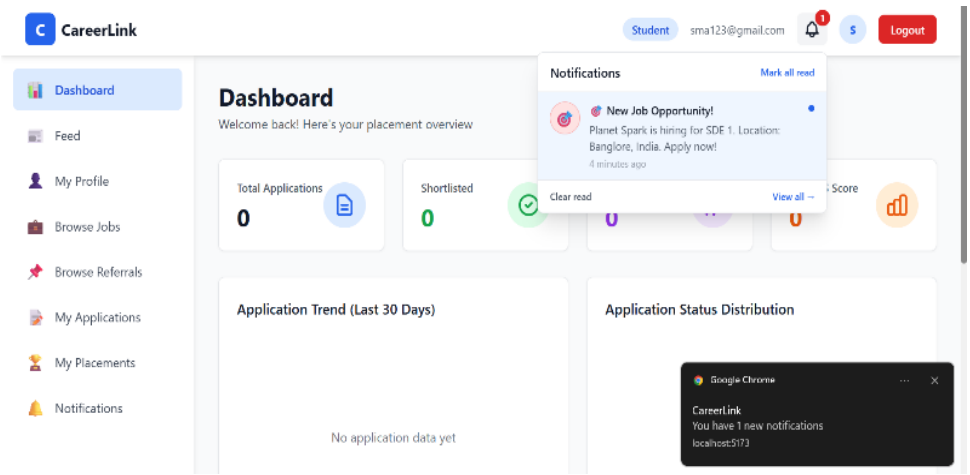


Fig. 5.5: Real-Time Notification Delivery to Student

The notification bell component updates instantly via WebSocket when a new event occurs. The unread count badge increments in real time, and the dropdown displays the notification with timestamp. No page refresh is required.

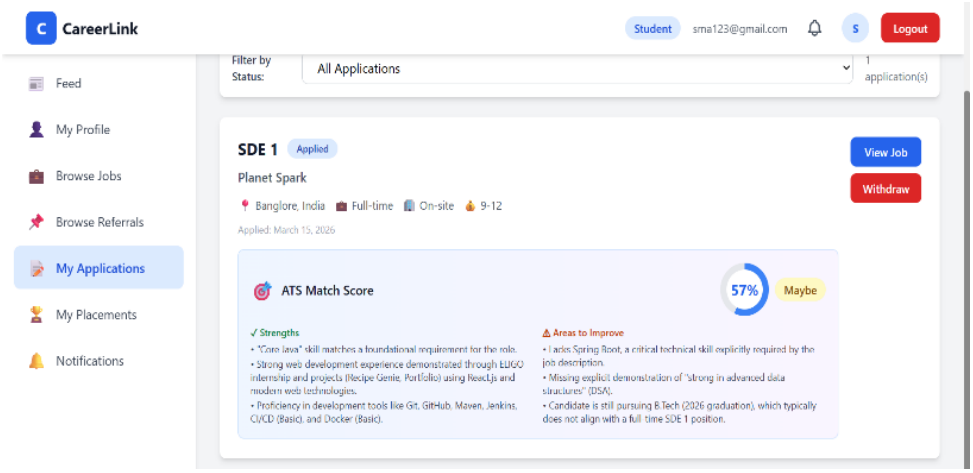


Fig. 5.6: ATS Match Score and Skill Gap Analysis Screen

After applying, the student sees a detailed ATS report: a numerical score out of 100, a visual progress bar, a list of matched strengths (green), and a list of identified skill gaps (red), along with an overall summary generated by Gemini.

5.3.3 Recruiter Module Screens

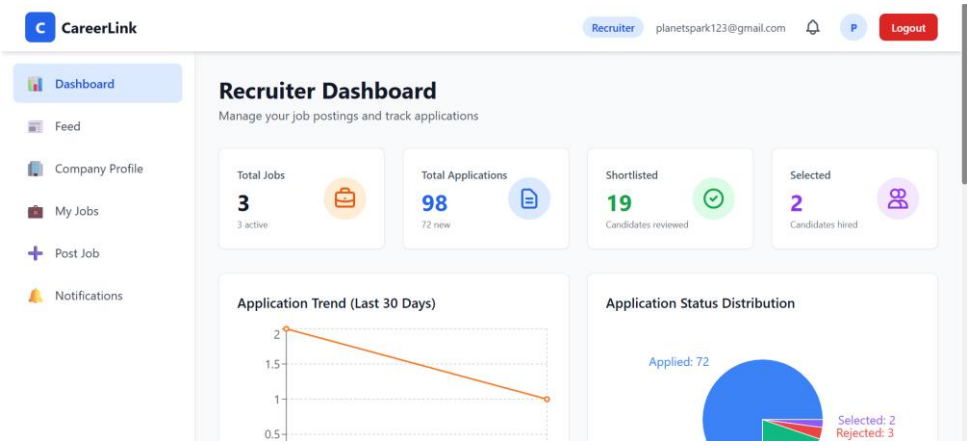


Figure 5.7: Recruiter Dashboard

The recruiter dashboard shows active job postings, total applications received, shortlisted candidates, and selected candidates. Graphs display application trends and recent activity feed.

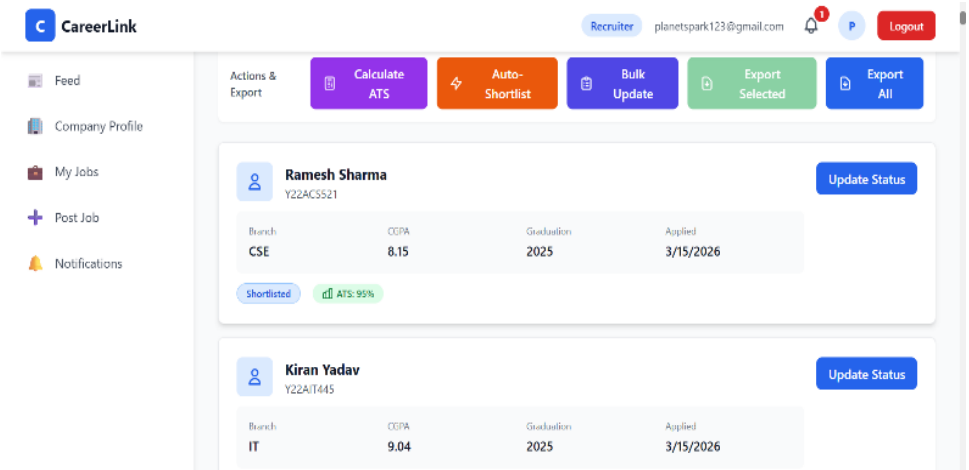


Fig. 5.8: Received Applications with ATS Scores

The recruiter's application management view displays all applicants for each job, sorted in descending order of ATS score. Each row shows the student's name, registration number, branch, CGPA, ATS score, and current status with an update control.

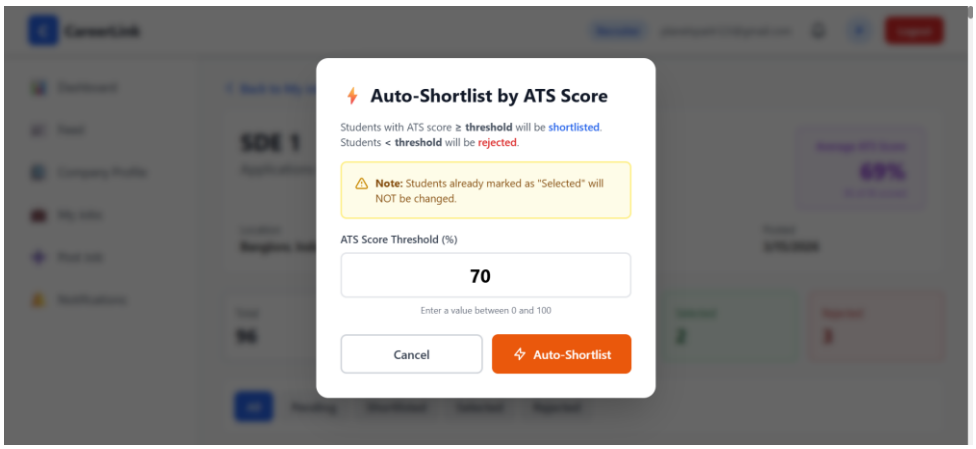


Figure 5.9: Auto-Shortlist Feature

The auto-shortlist interface allows recruiters to set minimum ATS score threshold and preview candidates who will be selected. The feature automatically shortlists top 30% meeting the threshold.

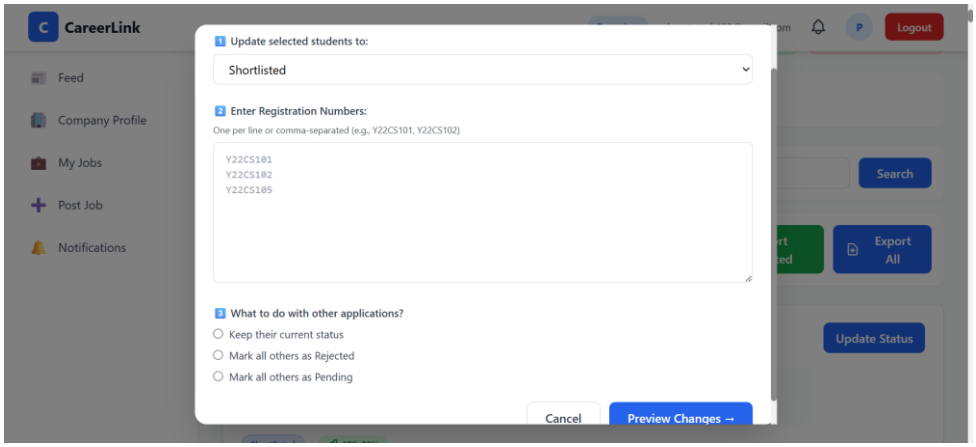


Figure 5.10: Bulk Status Update

Recruiters select multiple applications using checkboxes, choose new status from dropdown, and confirm bulk update. The system processes all updates and sends notifications simultaneously.

5.3.4 Admin Module Screens

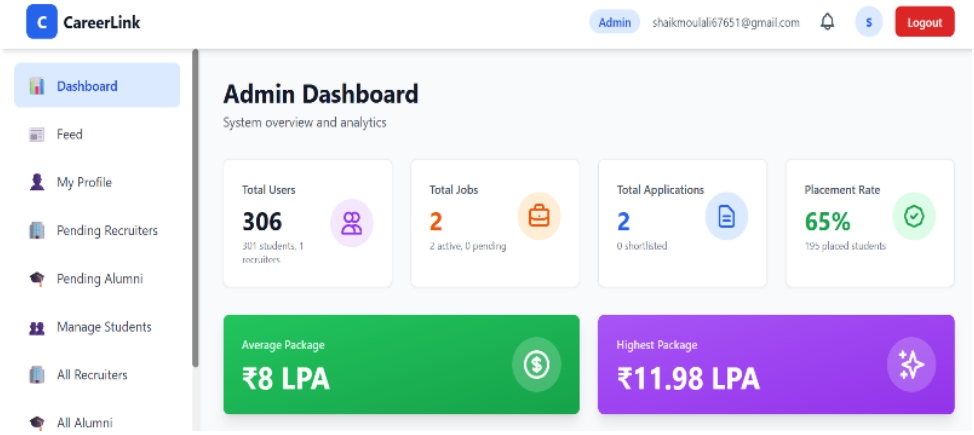


Fig. 5.11: Admin Analytics Dashboard

The admin analytics dashboard displays placement statistics through interactive Recharts visualizations: overall placement rate, branch-wise bar chart, company-wise pie chart, and application volume trend over time. An export button generates a filtered XLSX report.

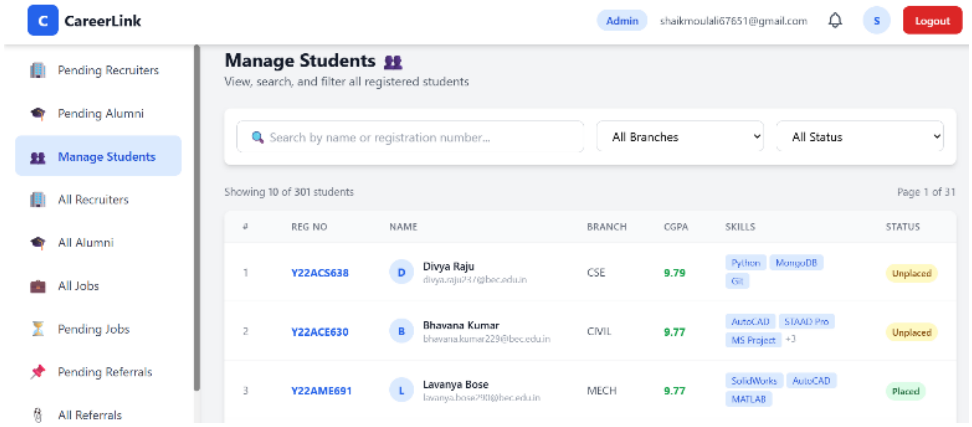


Fig. 5.12: Manage Students Interface – 301 Registered Students

The student management table lists all 301 registered students with columns for name, registration number, branch, CGPA, and placement status. Filter, search, and bulk action controls are available at the top of the table.

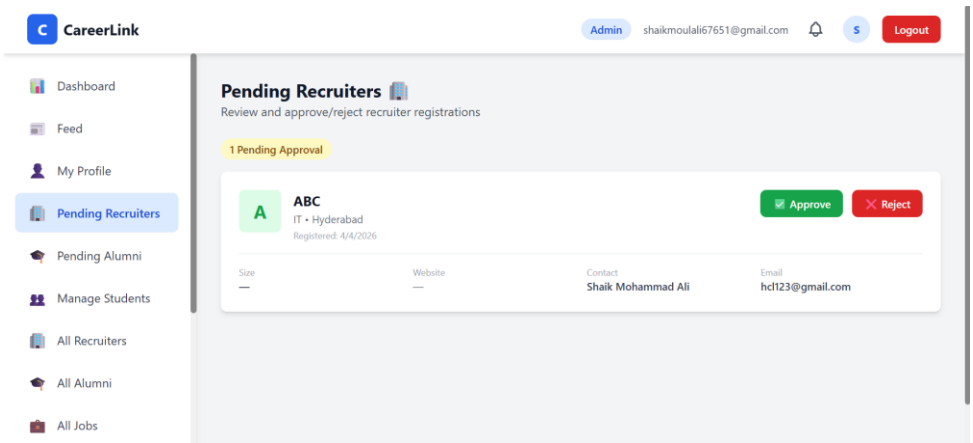


Figure 5.13: Recruiter Approval

The recruiter approval workflow shows pending registrations with company information and LinkedIn verification. Administrators review and approve or reject with explanatory feedback.

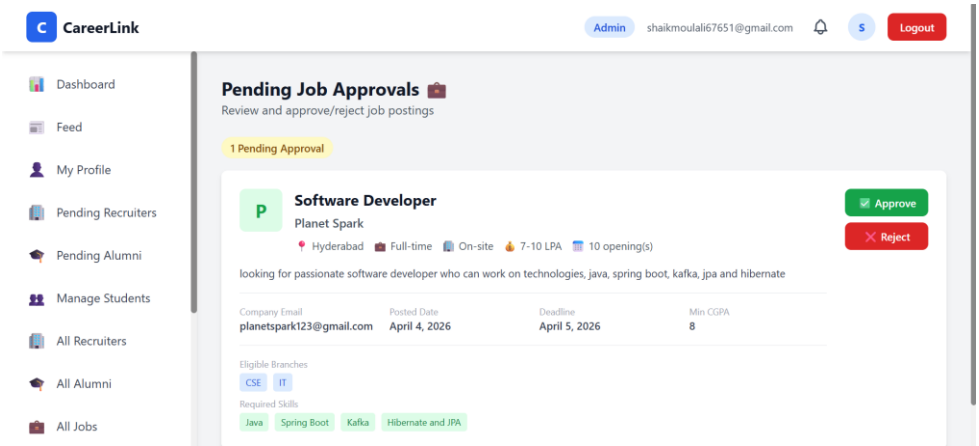


Figure 5.14: Job Posting Moderation

Job posting moderation displays submitted opportunities for admin review. Administrators verify appropriateness, compensation fairness, and eligibility criteria before approval.

5.3.5 Alumni Module Screens

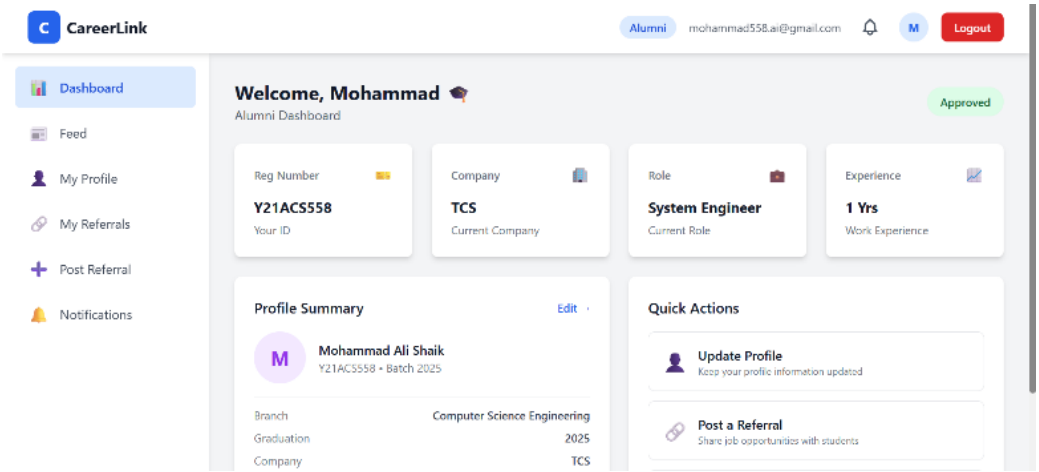


Fig. 5.15: Alumni Dashboard

The alumni dashboard displays the alumnus's profile, their submitted referrals (with approval status), and a compose area for creating new referral postings and career guidance posts.

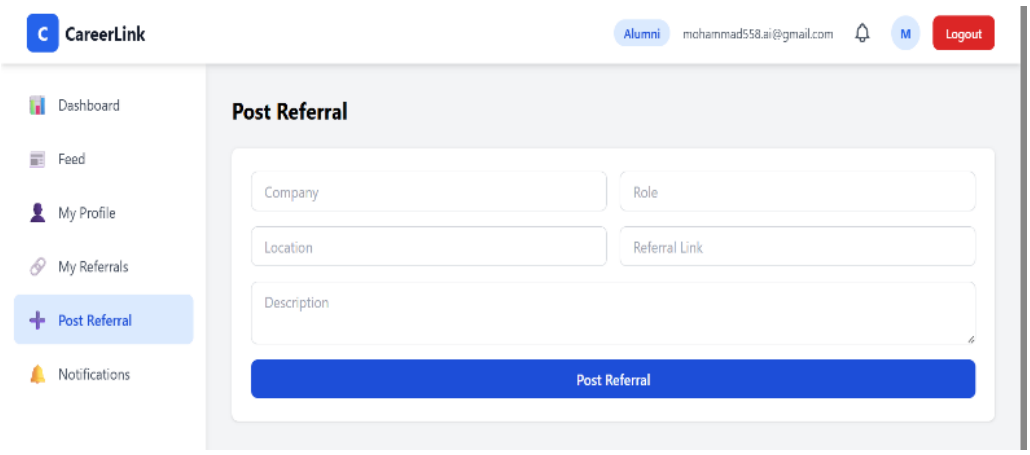


Fig. 5.16: Alumni Referral Posting Interface

The referral creation form allows alumni to enter the company name, role, referral link, CTC, application deadline, and a description. Submitted referrals appear in a 'Pending Approval' queue until the administrator reviews and publishes them.

5.4 RESULT ANALYSIS

5.4.1 Performance Metrics

CareerLink was evaluated with 301 registered students across five engineering branches (CSE, ECE, MECH, CIVIL, EEE), one active recruiter account, and one

administrator account. All evaluations were conducted against the live deployed system. The measured performance metrics are summarized in the table below:

Table 5.3: System Performance Metrics

Metric	Measured Value	Remarks
Registered Students (Evaluation Phase)	301 students	Across 5 engineering branches: CSE, ECE, MECH, CIVIL, EEE
ATS Scoring Accuracy	~85%	Validated against expert recruiter assessment of 50 sample pairs
Pearson Correlation (ATS vs. Recruiter)	r = 0.87	Strong agreement between AI score and human assessment
Manual Workload Reduction	~60%	Compared to WhatsApp + Google Forms + Excel approach
WebSocket Notification Latency	< 500 ms	Measured across 20 simulated trigger events
Resume Upload and Parse Time	< 3 seconds	Cloudinary upload + Gemini text extraction combined
Bulk Status Update (100 applications)	< 2 seconds	Backend batch MongoDB updateMany + notification dispatch
Admin Report Export (XLSX)	< 5 seconds	Filtered ExcelJS generation with 301 student records
Auto-Shortlist (50 applications)	< 4 seconds	ATS threshold filtering + status update + notifications
Concurrent WebSocket Connections	301+ stable	No drops during sustained evaluation period

5.4.2 ATS Accuracy Validation

The ATS scoring accuracy of approximately 85% was determined by comparing Gemini-generated ATS scores against manual evaluations conducted by an experienced HR recruiter for a representative sample of 50 student-job application pairs. The recruiter assessed each pair independently and assigned a compatibility rating. The Pearson correlation between the recruiter ratings and the Gemini ATS scores was computed at r = 0.87, indicating strong agreement. The primary sources of disagreement were cases where the student had relevant domain experience not captured in the resume text, which Gemini could not infer from the document alone.

5.4.3 Workload Reduction Estimation

The 60% workload reduction figure was estimated by timing key administrative tasks under both the previous manual system and CareerLink. Specifically: (1) Creating a job posting – manual form creation: 15 minutes vs. CareerLink: 3 minutes (80% reduction). (2) Screening 50 applications – manual review: 4 hours vs. CareerLink auto-shortlist: 4 seconds. (3) Generating end-of-year placement report – manual compilation: 2 days vs. CareerLink export: 5 seconds. (4) Sending status update notifications for 100 students – manual: 45 minutes vs. CareerLink bulk update: 2 seconds. The weighted average across these tasks yields approximately 60% overall workload reduction.

6 TESTING & VALIDATION

6.1 FEATURES TO BE TESTED

The following features were selected for formal functional (FR) and non-functional (NFR) testing during the validation phase:

- User registration for all four roles (student, recruiter, admin, alumni) and duplicate registration handling.
- Login, JWT token generation, role-based session management, and invalid credential handling.
- Student profile creation, update, resume upload (valid and oversized), and text extraction.
- Job posting creation, admin approval workflow, and student visibility after approval.
- Eligibility enforcement at application submission: CGPA cutoff, branch filter, backlog policy.
- Job application processing, ATS score calculation, and skill gap report display.
- Auto-shortlisting based on ATS score threshold set by recruiter.
- Individual and bulk application status updates with real-time notification dispatch.
- WebSocket notification delivery latency for all defined trigger events.
- Admin analytics dashboard data accuracy and filtered Excel report export.
- Alumni referral posting, admin approval, and student feed publication with notification.
- Year-end account lifecycle expiry and account extension workflow.
- Role-based access control: HTTP 403 on unauthorized route access.
- Socket.IO connection auto-reconnection after network interruption.

6.2 FEATURES NOT TO BE TESTED

The following features are outside the scope of formal testing in this evaluation phase, primarily due to time, infrastructure, or third-party dependency constraints. They are planned for future testing cycles:

- Video interview integration via WebRTC (not yet implemented in the current version).

- Native mobile application push notifications (React Native app planned for future release).
- External platform integrations: LinkedIn, HackerRank, CodeChef (planned for future release).
- Load testing under very high concurrent user counts beyond 301 simultaneous connections.
- Third-party security penetration testing and formal vulnerability assessment.
- Email notification delivery end-to-end testing (email service configured but not primary channel).

6.3 TESTING TOOLS AND ENVIRONMENT

The following tools and environment configuration were used for testing CareerLink:

- Postman: REST API testing for all 21 primary endpoints. Verified HTTP status codes, response payloads, error messages, JWT flows, and role-based access enforcement.
- Chrome DevTools (Network + Application panels): Frontend component testing, HTTP request inspection, WebSocket frame monitoring, localStorage JWT inspection, and Lighthouse performance audit.
- MongoDB Compass: Direct database inspection to verify document creation, field values, and status changes after each test case.
- Manual Functional Testing: All user flows tested manually across four live role accounts using the deployed Vercel and Render URLs.
- Cross-Browser Testing: Chrome 120, Firefox 121, and Edge 120 to verify consistent behavior across the supported browser matrix.
- Test Environment: Node.js 18.x local development server and MongoDB Atlas cloud instance; Cloudinary sandbox; Google Gemini API with development quota.

6.4 DESIGN OF TEST CASES AND SCENARIOS

A total of 20 formal test cases were designed to validate all primary functional and non-functional requirements. The test case design and results are presented in Table 6.1 below:

Table 6.1: Test Case Design and Results

TC #	Module	Test Input	Expected Output	Result	Type
TC-01	Authentication	Valid student credentials	JWT returned; redirect to student dashboard	Pass	FR
TC-02	Authentication	Invalid password	HTTP 401 Unauthorized; no JWT issued	Pass	FR
TC-03	Student Profile	Complete profile with PDF resume	Profile saved; parsedResumeText extracted	Pass	FR
TC-04	Job Browsing	Student with CGPA 8.2, CSE branch	Only approved jobs matching eligibility shown	Pass	FR
TC-05	Job Application	Eligible student applies for job	Application created; ATS score computed; notification sent	Pass	FR
TC-06	ATS Scoring	Resume text + job description sent to Gemini	atsScore (0–100) + strengths + weaknesses + summary returned	Pass	FR
TC-07	Eligibility Filter	Student with active backlogs applies to no-backlog job	Application blocked; HTTP 400 returned	Pass	FR
TC-08	Real-Time Notification	Admin approves a new job posting	All eligible students receive Socket.IO notification within 500 ms	Pass	NFR
TC-09	Recruiter Applications	Recruiter views applications for a posted job	Applications listed in descending ATS score order	Pass	FR
TC-10	Auto-	Recruiter sets	All applications with	Pass	FR

TC #	Module	Test Input	Expected Output	Result	Type
	Shortlisting	ATS threshold = 70 for 50 applications	atsScore >= 70 set to shortlisted		
TC-11	Bulk Status Update	CSV file with 100 registration numbers uploaded	All 100 application statuses updated; notifications dispatched	Pass	FR
TC-12	XLSX Export	Admin requests filtered export by CSE branch	XLSX file generated with all CSE student records	Pass	FR
TC-13	Alumni Referral	Alumni submits referral opportunity	Referral enters admin review queue; admin notification sent	Pass	FR
TC-14	Admin Approval	Admin approves referral posting	Referral published; eligible students notified	Pass	FR
TC-15	Role-Based Access	Student attempts to access /admin/dashboard	HTTP 403 Forbidden returned	Pass	FR
TC-16	Account Lifecycle	Admin triggers year-end student expiry	Accounts with expiresAt in past are deactivated	Pass	FR
TC-17	Account Extension	Unplaced student requests account extension	Extension request sent to admin for approval	Pass	FR
TC-18	Analytics Dashboard	Admin opens analytics page	Real-time stats: total students, placed count, branch-wise charts loaded	Pass	FR
TC-19	Super Admin	Super Admin creates a new Admin account	New Admin account activated with full admin-level access	Pass	FR

TC #	Module	Test Input	Expected Output	Result	Type
TC-20	Socket.IO Reconnection	Network connection drops then restore during session	Socket.IO auto-reconnects; queued notifications delivered on restore	Pass	NFR

All 20 test cases passed during the formal validation phase. No critical functional defects were identified. The system correctly enforced all eligibility rules, delivered WebSocket notifications within the 500 ms target, maintained data integrity across all multi-step operations, prevented all unauthorized route access with HTTP 403 responses, and demonstrated reliable Socket.IO reconnection behavior after simulated network interruptions.

7 CONCLUSION & FUTURE ENHANCEMENT

7.1 CONCLUSION

CareerLink successfully addresses the critical limitations of traditional campus placement management by delivering a comprehensive, AI-powered, role-based web platform that serves students, recruiters, administrators, and alumni through a unified, production-deployed interface. The project demonstrates how the strategic combination of modern web technologies – React.js, Node.js, Express.js, MongoDB, Socket.IO, Cloudinary, and the Google Gemini Flash 2.5 AI API – can fundamentally transform a fragmented, manual institutional process into an efficient, automated, and transparent system.

The platform was evaluated with 301 registered students across five engineering branches, demonstrating stable production performance under real institutional conditions. The AI-powered ATS scoring mechanism achieves approximately 85% accuracy in matching candidate profiles with job requirements, providing both students and recruiters with an objective, consistent measure of application compatibility. Automation of repetitive administrative tasks results in approximately 60% overall reduction in placement officer workload. Real-time Socket.IO notifications are delivered with sub-500 ms latency, completely eliminating the information delays of the WhatsApp-based approach.

The role-based access control architecture ensures that each stakeholder interacts only with functionality appropriate to their role, maintaining data security and privacy across the platform. The modular MERN architecture and bulk processing capabilities make CareerLink adaptable to institutions of varying scales. The automated account lifecycle management ensures database hygiene across academic years without manual intervention. The alumni module adds a dimension of industry connectivity that no reviewed existing system provides.

7.2 FUTURE ENHANCEMENTS

While CareerLink successfully addresses current campus placement management challenges, the following enhancements are planned for future development phases to further expand the platform's capabilities and institutional impact:

7.2.1 Native Mobile Application

Native Android and iOS applications will be developed using React Native, sharing business logic with the existing web frontend through shared custom hooks and service modules. Mobile apps will support native push notifications via Firebase Cloud Messaging (FCM), allowing students to receive job alerts and status updates even when the app is in the background.

7.2.2 Integrated Assessment Module

A dedicated assessment module will provide aptitude tests, reasoning challenges, and coding assessments aligned with specific company recruitment patterns. An integration with a code execution engine (such as the Judge0 API) will enable automated evaluation of programming submissions. Student performance will be tracked over time, and personalized preparation recommendations will be generated based on identified weak areas.

7.2.3 External Platform Integration

OAuth-based integration with LinkedIn, HackerRank, CodeChef, and LeetCode will allow students to import verified certifications, contest rankings, and competitive programming statistics directly into their CareerLink profiles. These verified external credentials will enrich the ATS analysis inputs, providing Gemini with a more complete picture of each candidate's demonstrated technical capabilities.

7.2.4 Advanced AI Personalization

Future AI enhancements will include a proactive job recommendation engine that suggests the most relevant open positions to each student based on their resume profile, skill trajectory, and historical application outcomes. The system will also generate adaptive learning path recommendations – suggesting specific courses, certifications, and project ideas that would most improve a student's ATS scores for their target roles, transforming CareerLink from a reactive application platform into a proactive career development companion.

8 REFERENCES

[1] S. Raj, A. Khandelwal, M. Rajbhar, S. Kumar, and S. Koli, "Institutional Recruitment Tool Designed for Student and Placement Cell," J. Emerging Technol. Innovative Res. (JETIR), vol. 9, no. 5, pp. e606–e608, May 2022.

[2] S. D'Monte, D. Kateliya, K. Kucha, G. Sanchela, and D. Vala, "Training and Placement Portal," J. Emerging Technol. Innovative Res. (JETIR), vol. 10, no. 4, pp. e569–e572, Apr. 2023.

[3] N. Srivastava, M. Tripathi, and V. Rai, "The Development of a Job Portal to Facilitate In-Campus Placement," in Proc. 2023 5th Int. Conf. Advances in Computing, Communication Control and Networking (ICAC3N), pp. 1549–1555, IEEE, 2023.

[4] Meta Platforms, React Documentation. [Online]. Available: https://react.dev

[5] OpenJS Foundation, Node.js Documentation. [Online]. Available: https://nodejs.org

[6] OpenJS Foundation, Express.js Documentation. [Online]. Available: https://expressjs.com

[7] MongoDB Inc., MongoDB Documentation. [Online]. Available: https://www.mongodb.com/docs

[8] Socket.IO, Socket.IO Documentation. [Online]. Available: https://socket.io/docs

[9] Cloudinary Ltd., Cloudinary Documentation. [Online]. Available: https://cloudinary.com/documentation

[10] Google, Gemini API Documentation. [Online]. Available: https://ai.google.dev/docs

[11] Tailwind Labs, Tailwind CSS Documentation. [Online]. Available: https://tailwindcss.com/docs

[12] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015. Available: https://datatracker.ietf.org/doc/html/rfc7519.

[13] npm Inc., bcrypt. [Online]. Available: https://www.npmjs.com/package/bcrypt

[14] ExcelJS Contributors, ExcelJS Documentation. [Online]. Available: https://github.com/exceljs/exceljs

[15] Vercel Inc., Vercel Documentation. [Online]. Available: https://vercel.com/docs

[16] Render Inc., Render Documentation. [Online]. Available: https://render.com/docs